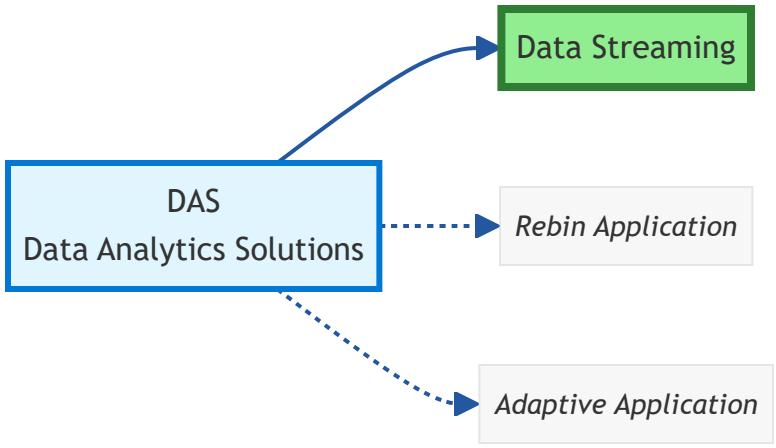
Data Streaming

Understand how the data stream delivers real-time access to comprehensive tester data.

> **NOTE**
>
> **DAS Definition:** A DAS (Data Analytics Solution) is a software application that subscribes to Archimedes data streams to analyze tester data and drive insights or actions.

This chapter focuses on the data streaming capabilities within the DAS ecosystem, as illustrated below:



This section covers:

1. DAS Overview - Understanding Data Analytic Solutions
2. Advanced DAS Concepts
3. Data Format and Structure
4. Implementing a DAS Listener
5. URL Reservation

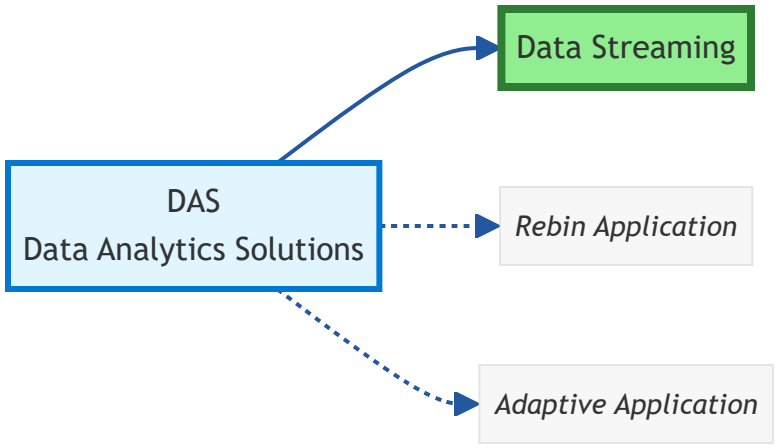
This documentation is part of the Teradyne Archimedes Reference Architecture series.

Data Streaming

Understand how the data stream delivers real-time access to comprehensive tester data.

> **NOTE**
>
> **DAS Definition:** A DAS (Data Analytics Solution) is a software application that subscribes to Archimedes data streams to analyze tester data and drive insights or actions.

This chapter focuses on the data streaming capabilities within the DAS ecosystem, as illustrated below:



This section covers:

1. DAS Overview - Understanding Data Analytic Solutions
2. Advanced DAS Concepts
3. Data Format and Structure
4. Implementing a DAS Listener
5. URL Reservation

This documentation is part of the Teradyne Archimedes Reference Architecture series.

Understanding DAS (Data Analytics Solution) and REST Communication

What is a DAS?

A **DAS** (Data Analytics Solution) is a system designed to **receive, process, and manage data** sent by other applications — in this case, by **AMP**.
AMP communicates with each DAS through **HTTP** using the **REST** protocol.

The DAS acts as a **listener service** exposing one or more **REST endpoints**.
Each endpoint can receive or respond to HTTP messages such as `POST` (to send data) or `GET` (to retrieve information).

How DAS and AMP Communicate

Communication between **AMP** and **DAS** follows a REST-based message exchange model:

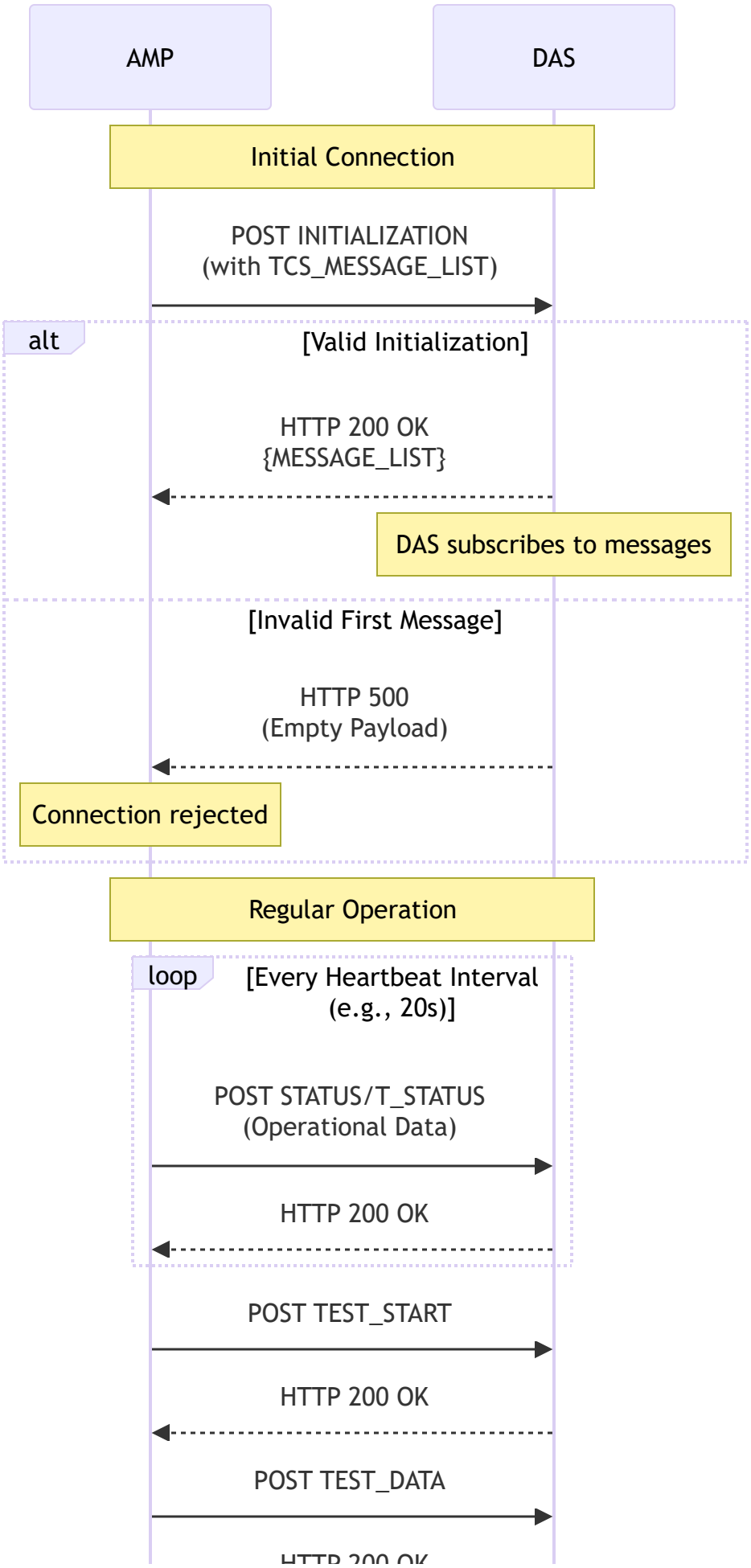
1. **AMP sends data** to one or more DAS instances using HTTP.
2. Each **DAS exposes REST endpoints** such as:

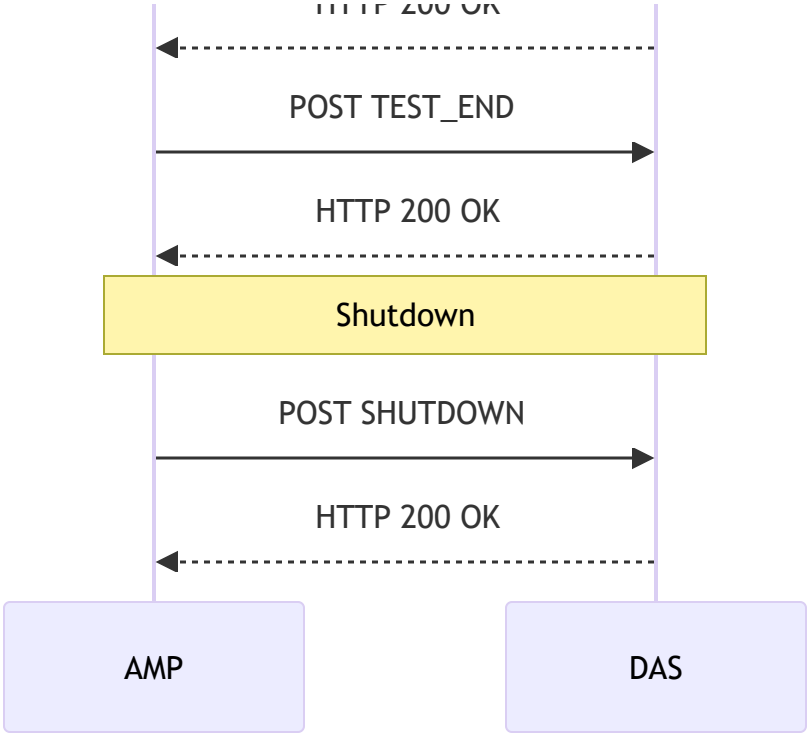
```
http://127.0.0.1:3000/tems/TEST_CELL/{hostname}/STATUS
http://127.0.0.1:1805/TRV/TEST_CELL/{hostname}/STATUS
http://10.100.100.1:5002/AdaptiveDAS/TEST_CELL/{hostname}/STATUS
```

3. **AMP uses the `POST` method** to send JSON-formatted messages containing operational data.
4. **DAS validates and processes** incoming messages, then returns an HTTP status code and, if valid, a `MESSAGE_LIST` JSON response (the DAS subscriptions).

Communication Flow Diagram

The following diagram illustrates the typical message exchange between AMP and DAS:


AMP
DAS
Initial Connection
POST INITIALIZATION
(with TCS_MESSAGE_LIST)
alt
[Valid Initialization]
HTTP 200 OK
{MESSAGE_LIST}
DAS subscribes to messages
[Invalid First Message]
HTTP 500
(Empty Payload)
Connection rejected
Regular Operation
loop
[Every Heartbeat Interval
(e.g., 20s)]
POST STATUS/T_STATUS
(Operational Data)
HTTP 200 OK
POST TEST_START
HTTP 200 OK
POST TEST_DATA



Initialization Sequence

When AMP starts communication with a DAS, the **first message** must always be an `INITIALIZATION` message.

If the DAS receives any message other than `INITIALIZATION` as the first message, it **must reject the request** and respond with:

- An **HTTP error code** (e.g., `500 Internal Server Error`), and
- An **empty payload**.

Example of a Rejected First Message

```
HTTP/1.1 500 Internal Server Error
Content-Length: 0
```

Example of an INITIALIZATION Message (from AMP)

The **INITIALIZATION** message defines the start of communication between AMP and DAS.
It includes system information, supported message types, and operational parameters.

```
{
  "TIME_STAMP": "2025-10-29T16:00:13.9855907Z",
  "TCS_VENDOR": "Teradyne",
  "TCS_VERSION": "TEMS_06.10.06-42",
```

```
  "TEMS_VERSION": "0.80",
  "TCS_MESSAGE_LIST": [
    "INITIALIZATION",
    "SHUTDOWN",
    "STATUS",
    "T_STATUS",
    "USER_ACCOUNT",
    "DTR",
    "IGXL_DATA",
    "MST_DATA",
    "QUALITY_MONITOR_DATA",
    "TEST_END",
    "TESTER_OS",
    "TEST_PROGRAM_LOAD",
    "TEST_START",
    "CONFIGURATION",
    "LOCATION",
    "BINNAME",
    "RESULTFILE",
    "LOT_END",
    "LOT_START",
    "SUBLOT_END",
    "SUBLOT_START",
    "HALT",
    "LICENSE",
    "PERIPHERAL",
    "ALARM",
    "MAINTENANCE",
    "MAINTENANCE_DATA",
    "CZ_DATA_POINT",
    "CZ_END",
    "CZ_SETUP",
    "CZ_START",
    "CZ_START_POINT",
    "CZ_SUMMARY",
    "TEST_DATA"
  ],
  "TCS_CONTROL_LIST": [],
  "TCS_LOST_MESSAGES": 55,
  "TCS_STATUS_FREQUENCY": 60,
  "IS_PRIMARY_DAS": true,
  "TCS_COMPRESSION_TYPES": []
}
```


> `TCS_MESSAGE_LIST` lists the message types AMP supports and may send.

DAS Response Behavior

After receiving a **valid** message from AMP, the **DAS** must:

1. **Return an HTTP status code**:
    - `200 OK` → Message accepted
    - `400 Bad Request` → Invalid or malformed message
    - `500 Internal Server Error` → Invalid message sequence or system error
2. **For valid messages**, include in the response body a JSON object containing the `MESSAGE_LIST`: the list of message types the **DAS subscribes to** (i.e., wants to receive).

Example of a Valid DAS Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "MESSAGE_LIST": [
    "INITIALIZATION",
    "SHUTDOWN",
    "STATUS",
    "T_STATUS",
    "ALARM",
    "CONFIGURATION",
    "USER_ACCOUNT",
    "TESTER_OS",
    "TEST_PROGRAM_LOAD",
    "LOT_START",
    "LOT_END",
    "SUBLOT_START",
    "SUBLOT_END",
    "TEST_START",
    "TEST_END",
    "DTR",
    "HALT",
    "TEST_DATA",
    "PERIPHERAL",
    "MAINTENANCE",
    "MAINTENANCE_DATA",
    "LICENSE",
    "IGXL_DATA",
    "MST_DATA",
    "BINNAME",
    "RESULTFILE",
```

```
    "CZ_SETUP",
    "CZ_END",
    "CZ_DATA_POINT",
    "CZ_START",
    "CZ_START_POINT",
    "CZ_SUMMARY",
    "QUALITY_MONITOR_DATA",
    "LOCATION"
  ]
}
```

Example of an Error Response (Empty Payload)

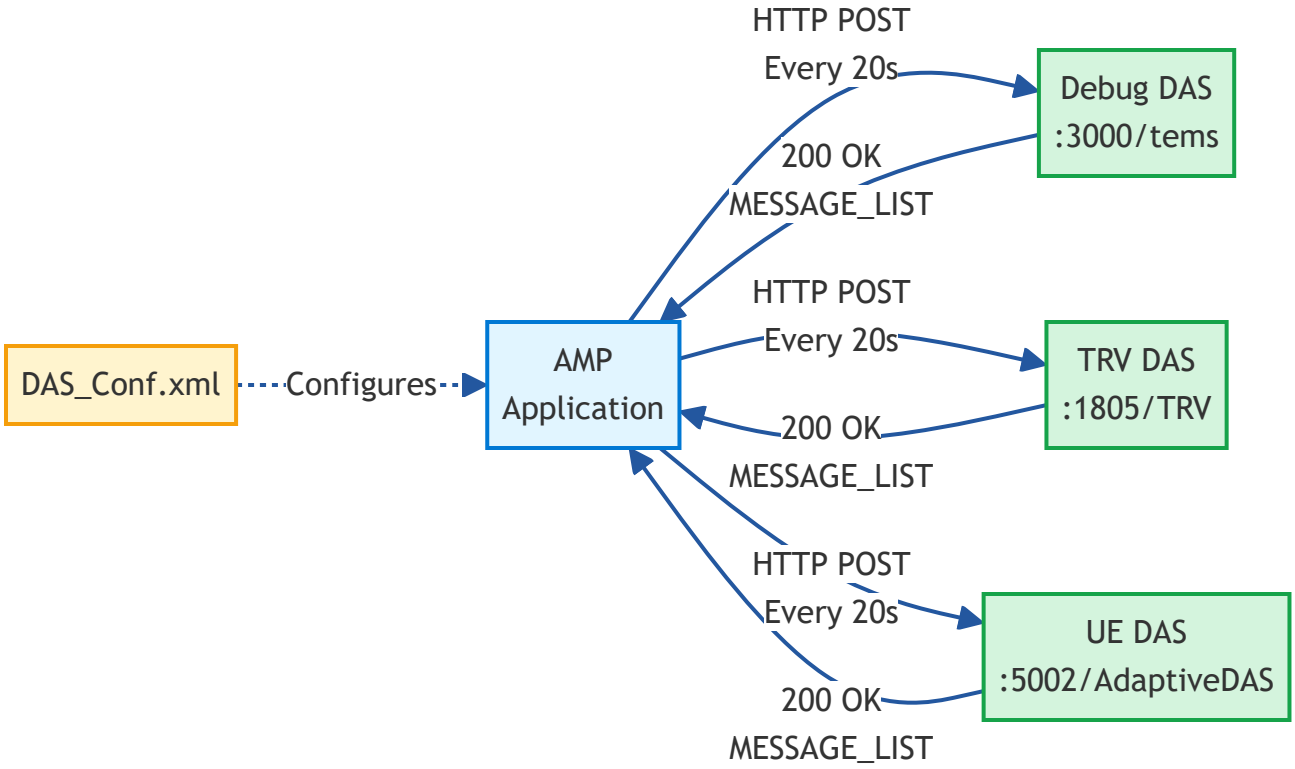
```
HTTP/1.1 500 Internal Server Error
Content-Length: 0
```

The DAS Configuration File (`DAS_Conf.xml`)

All DAS instances used by AMP are defined in the configuration file **`DAS_Conf.xml`**.
This file specifies each DAS endpoint, operational limits, and heartbeat intervals.

Architecture Overview

The following diagram shows how AMP connects to multiple DAS instances as configured in `DAS_Conf.xml`:



Example

```xml
<?xml version="1.0" encoding="utf-8"?>
<DASConfig xmlns:xsd="http://www.w3.org/2001/XMLSchema"
           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
           xmlns="http://www.teradyne.com">
  <DAS>
    <URL>http://127.0.0.1:3000/tems/</URL>
    <MaxMsg>20</MaxMsg>
    <Name>Debug DAS</Name>
    <Enable>true</Enable>
    <Specification>S_08</Specification>
  </DAS>
  <DAS>
    <URL>http://127.0.0.1:1805/TRV/</URL>
    <MaxMsg>1</MaxMsg>
    <Name>TRV</Name>
    <Enable>true</Enable>
    <Specification>S_08</Specification>
  </DAS>
  <DAS>
    <URL>http://10.100.100.1:5002/AdaptiveDAS/</URL>
    <MaxMsg>1</MaxMsg>
    <Name>UE</Name>
    <Enable>true</Enable>
    <Specification>S_08</Specification>
```

```
  </DAS>
  <HeartBeat>20</HeartBeat>
  <T_Status>
    <HeartBeat>25</HeartBeat>
    <IdleThreshold>60</IdleThreshold>
  </T_Status>
</DASConfig>
```

Understanding the Heartbeat

The **heartbeat** defines the interval (in seconds) at which **AMP automatically sends messages** to all enabled DAS instances listed in `DAS_Conf.xml`.

For example:

- `<HeartBeat>20</HeartBeat>` → AMP sends a message to each DAS every **20 seconds**
- `<T_Status><HeartBeat>25</HeartBeat></T_Status>` → Specific interval for test status updates

This ensures:

- Continuous synchronization between AMP and each DAS
- Periodic operational updates
- Early detection of communication or system errors

Summary

Concept	Description
DAS	Data Analytics Solution — receives and processes HTTP messages from AMP
AMP	Analytics Management Platform — application that sends JSON-formatted messages to configured DAS endpoints
INITIALIZATION	The first message AMP must send to start communication
Rejection Rule	If the first message is not `INITIALIZATION`, DAS must return HTTP 500 with no payload
DAS Response	On success, DAS responds with `200 OK` and a `MESSAGE_LIST` JSON body representing its subscription
Error Behavior	If an error occurs, the DAS must return only the HTTP code — no payload

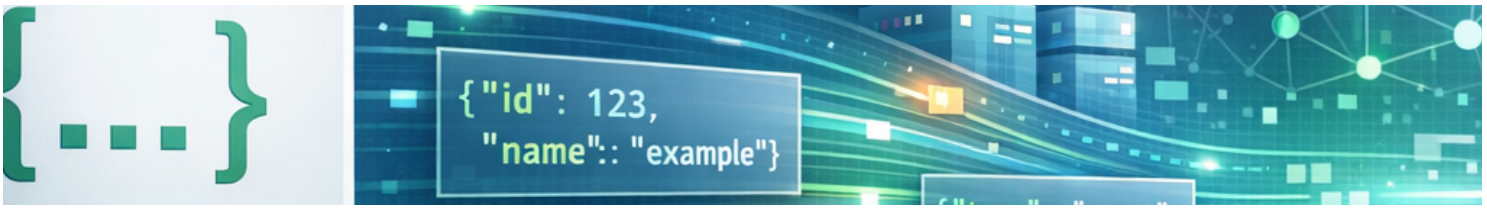
Concept	Description
Configuration	Defined in `DAS_Conf.xml` with URLs, heartbeat, and operational parameters
Heartbeat	Defines how often AMP sends messages to each DAS instance

---

✅ Key Takeaways

- The **first message** from AMP must always be `INITIALIZATION`; otherwise, the DAS rejects it with **HTTP 500** and **no payload**.
- Upon a **valid** message, the **DAS responds** with **200 OK** and returns its **`MESSAGE_LIST`** (the messages it subscribes to).
- **AMP message content** varies by type; only `INITIALIZATION` includes `TCS_MESSAGE_LIST`.
- `DAS_Conf.xml` defines endpoints and **heartbeat** intervals that drive periodic communication.

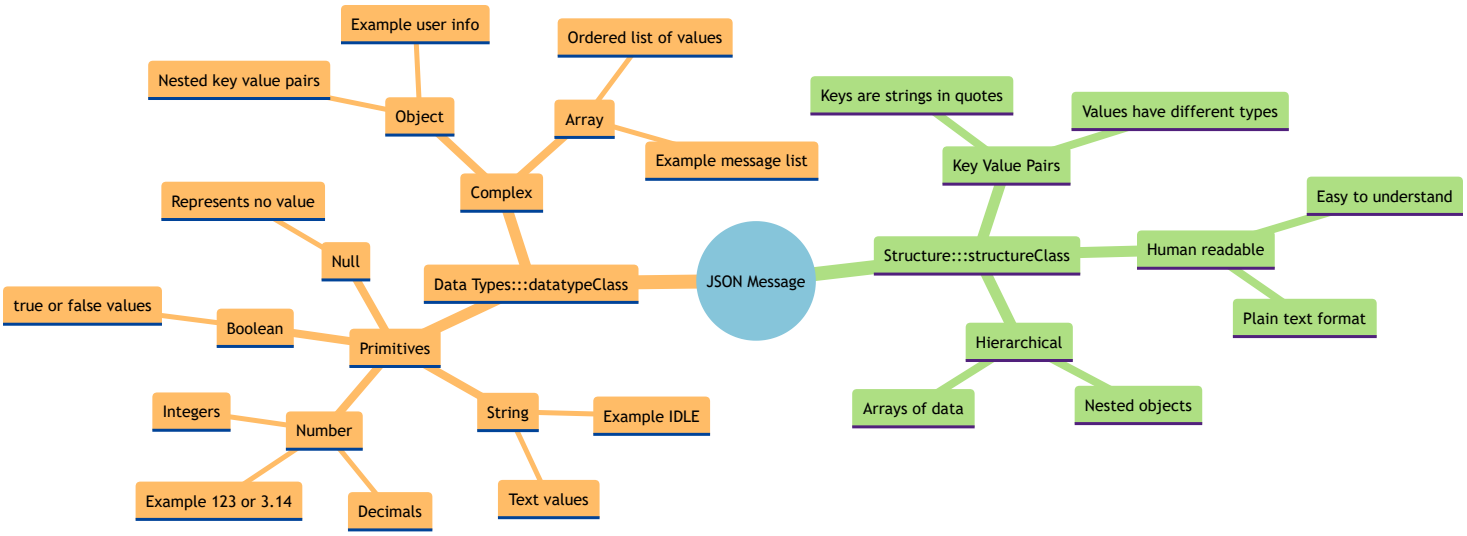
This documentation is part of the Teradyne Archimedes Reference Architecture series.



Introduction: Understanding JSON Messages

JSON (JavaScript Object Notation) is a lightweight and widely used data format for exchanging information between systems such as **AMP** and **DAS**.
It is structured in a simple key–value format that makes it both **human-readable** and **machine-parsable**.

What is a JSON Message? (High-Level Overview)

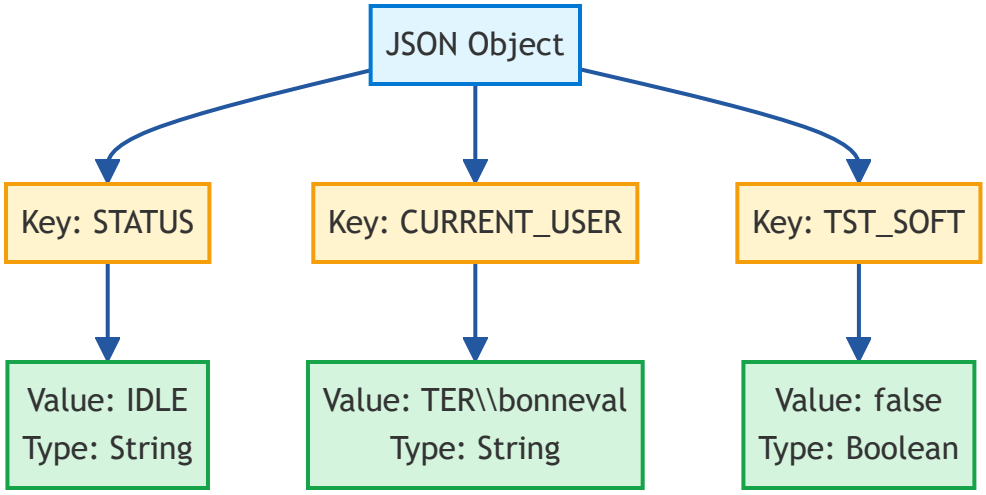


Basic Structure

A JSON message consists of:

- **Keys** (also called *fields*), written as text inside quotes.
- **Values**, which can be text, numbers, booleans, arrays, or even nested objects.

Visual Structure



Example

```
{
  "STATUS": "IDLE",
  "CURRENT_USER": "TER\\bonneval",
  "TST_SOFT": false
}
```

In this example:

- `"STATUS"` is a **key**, and `"IDLE"` is its **value**.
- `"CURRENT_USER"` is another key whose value is the logged user.
- `"TST_SOFT": false` means the test software is currently inactive.

Nested and Array Values

JSON can contain **arrays** (lists) and **nested objects**.
Arrays are surrounded by square brackets `[]`, and objects by curly braces `{}`.

Example with a message list:

```
{
  "MESSAGE_LIST": [
    "INITIALIZATION",
    "STATUS",
    "TEST_START",
    "TEST_END"
  ]
}
```

Here, `MESSAGE_LIST` is a key whose value is an array of strings.

Reading JSON in Practice

To read JSON data:

1. **Identify the top-level keys** – these describe the main categories of information (e.g., `TIME_STAMP`, `STATUS`, `MESSAGE_LIST`).
2. **Inspect the values** – determine what each represents (text, number, list, etc.).
3. **Follow hierarchy** – if a value is an object, open its `{}` section and repeat the process.
4. **Ignore order** – JSON fields are unordered; their sequence doesn't matter.

In the DAS Context

In the AMP–DAS communication:

- **AMP** sends messages formatted in JSON.
- **DAS** reads these messages, checks their **keys and values**, and responds with its own JSON structure.

Each message type (for example, `INITIALIZATION`, `STATUS`, or `TEST_END`) may include different fields, but they all follow the same readable JSON format.

Common Message Types

The following table shows the most common message types exchanged between AMP and DAS:

Message Type	Direction	Purpose	Key Fields
`INITIALIZATION`	AMP → DAS	Start communication	`TCS_MESSAGE_LIST`, `TCS_VENDOR`, `TCS_VERSION`
`STATUS`	AMP → DAS	System status update	`STATUS`, `CURRENT_USER`, `TST_SOFT`
`TEST_START`	AMP → DAS	Test execution started	`LOT_ID`, `SUBLOT_ID`, `TEST_PROGRAM`
`TEST_DATA`	AMP → DAS	Test results data	`TEST_NUMBER`, `RESULTS`, `BIN`
`TEST_END`	AMP → DAS	Test execution ended	`FINAL_BIN`, `DURATION`
`SHUTDOWN`	AMP → DAS	Close connection	`REASON`
`MESSAGE_LIST`	DAS → AMP	DAS subscriptions	`MESSAGE_LIST` (array)

Example: Complete STATUS Message

```
{
  "TIME_STAMP": "2025-12-17T14:23:45.1234567Z",
  "MESSAGE_TYPE": "STATUS",
```

```
  "STATUS": "TESTING",
  "CURRENT_USER": "TER\\bonneval",
  "TST_SOFT": true,
  "CELL_ID": "TEST_CELL_01",
  "PROGRAM_NAME": "PROGRAM_X_V1.2",
  "HEARTBEAT_COUNT": 142
}
```

Example: TEST_DATA Message with Nested Data

```
{
  "TIME_STAMP": "2025-12-17T14:24:10.5678901Z",
  "MESSAGE_TYPE": "TEST_DATA",
  "DEVICE_ID": "DEV_12345",
  "TEST_NUMBER": 1523,
  "RESULTS": {
    "VOLTAGE": 3.3,
    "CURRENT": 0.125,
    "TEMPERATURE": 25.4,
    "PASS": true
  },
  "BIN": 1,
  "SITE": 0
}
```

Going further

JSON Data Types Quick Reference

Type	Example	Description
String	`"IDLE"`, `"TEST_CELL_01"`	Text enclosed in double quotes
Number	`123`, `3.14`, `-42`	Integer or decimal values
Boolean	`true`, `false`	Logical true/false values
Array	`["A", "B", "C"]`	Ordered list of values
Object	`{"key": "value"}`	Nested key-value pairs
Null	`null`	Represents no value

Validation Tips

When working with JSON messages in DAS:

1. **Validate structure** – Ensure the message has all required keys
2. **Check data types** – Verify values match expected types (string, number, etc.)
3. **Handle missing fields** – Use default values or reject invalid messages
4. **Parse carefully** – Use proper JSON parsing libraries to avoid errors
5. **Log errors** – Record malformed messages for debugging

Example: JSON Parsing in C#

```
using System.Text.Json;

// Parse JSON string to object
var jsonString = @"{""STATUS"": ""IDLE"", ""TST_SOFT"": false}";
var doc = JsonDocument.Parse(jsonString);

// Access values
string status = doc.RootElement.GetProperty("STATUS").GetString();
bool testSoft = doc.RootElement.GetProperty("TST_SOFT").GetBoolean();

Console.WriteLine($"Status: {status}, Test Software: {testSoft}");
```

Example: JSON Parsing in Python

```
import json

# Parse JSON string to dictionary
json_string = '{"STATUS": "IDLE", "TST_SOFT": false}'
data = json.loads(json_string)

# Access values
status = data["STATUS"]
test_soft = data["TST_SOFT"]

print(f"Status: {status}, Test Software: {test_soft}")
```

Additional Resources

If you want to learn more about JSON

Understanding JSON is key to interpreting and debugging the communication between AMP and the Data Analytics Solution (DAS).

This documentation is part of the Teradyne Archimedes Reference Architecture series.

What Is a DAS Listener?

The **DAS Listener** is a REST-based server component used in the **Data Analytics Solution (DAS)** architecture. It is responsible for **receiving, validating, and responding to commands** from external systems — such as test programs, orchestration frameworks, or monitoring tools.

Understanding REST

REST (Representational State Transfer) is a way for programs to communicate over the web using simple HTTP requests. A client sends a message (usually in **JSON** format) to a specific URL (an "endpoint"), and the server replies with a result.

The communication pattern:

- The URL is the endpoint address
- The JSON payload contains the data
- The server responds with confirmation or error information

🔌 REST Communication in DAS

All DAS communication happens via **HTTP POST** requests using one of the following URL formats:

Accepted URL Patterns

1. **Basic Command:**

   ```
   <dasURL>/TESTCELL/<hostname>/<messagename>
   ```

2. **With Lot Context:**

   ```
   <dasURL>/TESTCELL/<hostname>/LOTID/<lotid>/<messagename>
   ```

3. **With Lot and Sublot Context:**

   ```
   <dasURL>/TESTCELL/<hostname>/LOTID/<lotid>/SUBLOT/<sublotid>/<messagename>
   ```

> Each message name corresponds to a command (e.g., `INITIALIZATION`, `TEST_START`, `STATUS`).

⏱ Required JSON Payload Fields

All messages **must** include at least:

- `"TIME_STAMP"`: an ISO 8601 UTC timestamp.

Example:

```
{
  "TIME_STAMP": "2025-06-30T09:15:00Z"
}
```

Initialization Phase

The **first message** sent by any test cell must be `INITIALIZATION`.
If this is missing or another command is sent first, the DAS Listener will reject the message with an error (`403` or `500`).

This message must also include a list of messages the test cell plans to send (aka the **subscription list**).

Example of Subscribed Messages

```
[
  "STATUS", "INITIALIZATION", "LICENSE", "T_STATUS", "SHUTDOWN", "HALT", "ALARM",
"LOCATION", "USER_ACCOUNT",
  "DTR", "IGXL_DATA", "MST_DATA", "QUALITY_MONITOR_DATA", "TEST_END",
"TESTER_OS", "TEST_PROGRAM_LOAD",
  "TEST_START", "CONFIGURATION", "BINNAME", "RESULTFILE", "LOT_END", "LOT_START",
"SUBLOT_END", "SUBLOT_START",
  "PERIPHERAL", "MAINTENANCE", "MAINTENANCE_DATA", "CZ_DATA_POINT", "CZ_END",
"CZ_SETUP", "CZ_START",
  "CZ_START_POINT", "CZ_SUMMARY", "TEST_DATA", "ADAPTIVE_CHANGE", "PRE_TEST_END"
]
```

The server will **echo back** this list in each response as confirmation.

Message Categories and URL Types

Type 1: Common Messages

URL Pattern: `<dasURL>/TESTCELL/<hostname>/<messagename>`

System & Configuration

Data Collection & Monitoring

```
%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#E8F4F8','primaryTextColor':'#00000
mindmap
  root((Data Collection & Monitoring))
    IG-XL Data
      IGXL_DATA
      QUALITY_MONITOR_DATA
    MST Data
      MST_DATA
    Test Results
      BINNAME
      RESULTFILE
    Monitoring
      T_STATUS
      PERIPHERAL
    Maintenance
      MAINTENANCE
      MAINTENANCE_DATA
```

Type 2: Lot Context

URL Pattern: `<dasURL>/TESTCELL/<hostname>/LOTID/<lotid>/<messagename>`

```
%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#F0F9E8','primaryTextColor':'#00000
mindmap
  root((TESTCELL/hostname/LOTID/lotid/messagename))
    LOT_START
    LOT_END
```

Type 3: Test & Sublot Context

URL Pattern: `<dasURL>/TESTCELL/<hostname>/LOTID/<lotid>/SUBLOT/<sublotid>/<messagename>`

```
%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#FFF4E6','primaryTextColor':'#00000
mindmap
  root((TESTCELL/hostname/LOTID/lotid/SUBLOT/sublotid/messagename))
    Test Results
      TEST_START
      TEST_END
```

```
    TEST_DATA
    DTR
  Sublot Management
    SUBLOT_START
    SUBLOT_END
  Test Program Behavior
    PRE_TEST_END
    ADAPTIVE_CHANGE
  Characterization
    CZ_SETUP
    CZ_START
    CZ_END
    CZ_DATA_POINT
    CZ_START_POINT
    CZ_SUMMARY
```

Message Lifecycle

1. Client sends `INITIALIZATION` message → DAS Listener returns `200 OK` + subscription list.
2. Client sends follow-up messages (e.g., `STATUS`, `TEST_START`) → Server echoes back subscription list.
3. If a message is invalid or not expected → Server replies with appropriate error.

HTTP Behavior

Method	Purpose	Notes
`POST`	Send message	✅ Only method supported
`GET`	❌ Not allowed	Will return `400 Bad Request`

Common HTTP Responses

Code	Meaning
`200`	Success – message processed
`400`	Bad Request – invalid format or method
`403`	Forbidden – not initialized
`500`	Internal Server Error – unexpected

This documentation is part of the Teradyne Archimedes Reference Architecture series.

DAS Listener Reference Architectures

The **Data Analytics Solution (DAS)** is a key component used to listen to and process structured data transmitted from external systems such as test programs, diagnostic tools, or orchestration services.

In a typical test flow, the DAS Listener acts as an intermediary between a data-emitting process (e.g., AMP, IG-XL, or a Python test harness) and a backend that stores, analyzes, or routes this information further.
Its primary responsibility is to **receive, deserialize, log, and optionally respond to data messages**, often formatted in JSON.

Purpose of the DAS

DAS Listeners are generally used to:

- Receive **real-time structured data** (commonly JSON)
- Parse incoming data and validate schemas
- Route messages to local logs, databases, message queues, or cloud endpoints
- Trigger application logic based on received content

This architecture decouples the data emission logic from data handling, enabling greater flexibility, scalability, and language independence.

For each implementation, we provide:

- Source code and architecture overview
- Setup instructions and dependencies
- Examples of test data and logging behavior

Available DAS Implementations

The following implementations demonstrate how to build and use a DAS Listener in different programming environments.
Each link points to a dedicated RA (Reference Architecture) detailing the architecture, code structure, and setup instructions.

DAS Implementation accross Languages

C#

C++

Java - Coming Soon

PowerShell

Python

This documentation is part of the Teradyne Archimedes Reference Architecture series.

How to implement a DAS Listener in C#

Reference : RA 468

Welcome to the documentation for the **RA_0468 DAS Listener**.
This reference architecture demonstrates how to build a responsive Data Analytics Solution (DAS) that listens for messages from a test program and processes them dynamically.

General Technical Info

Component	**Version**
Language	*C#*
Framework	.NET 8
Environment	Windows 10
IGXL	> 10.30.10
MST	> 2016C
IDE	Visual Studio Code / Visual Studio 2022

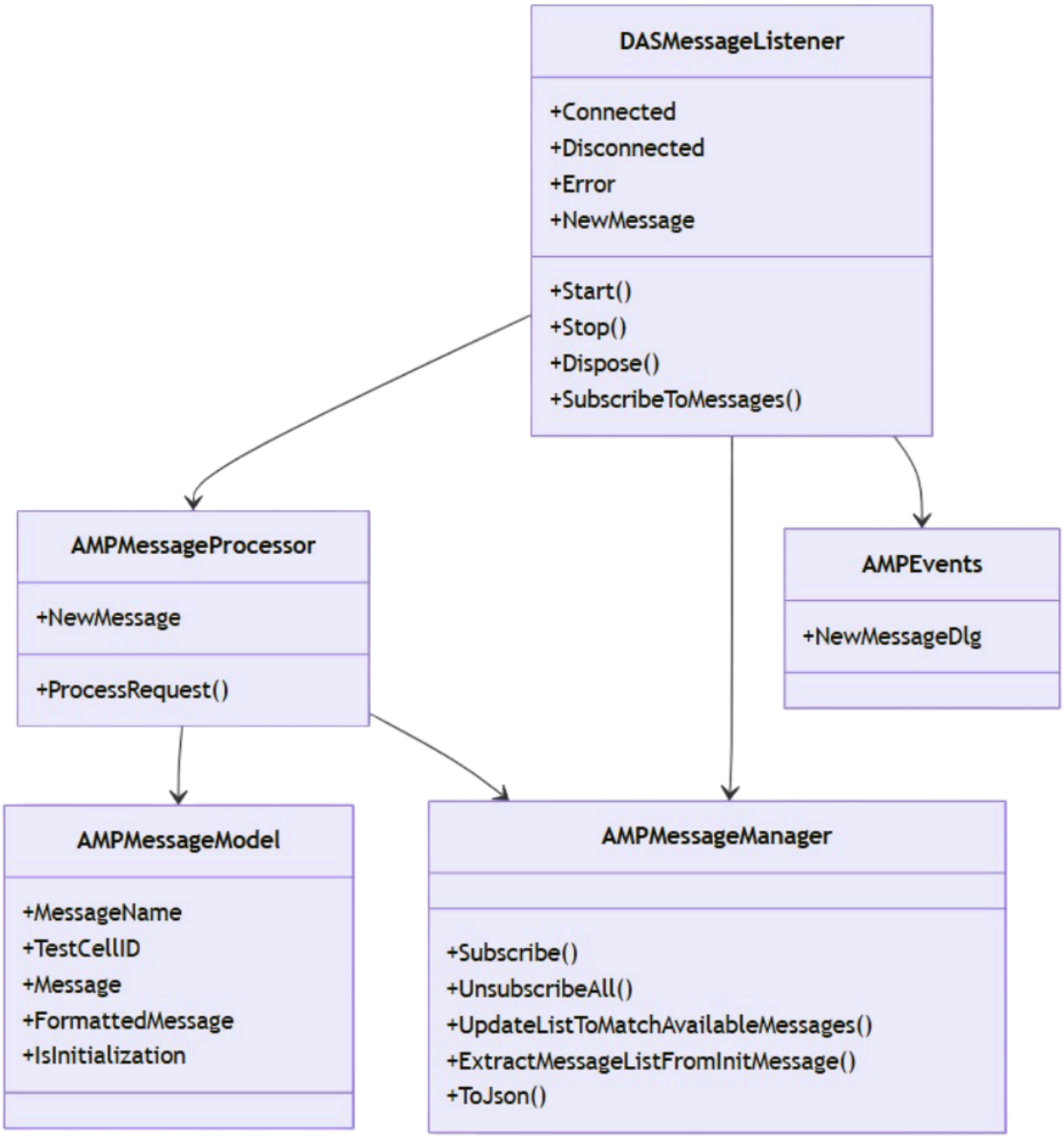
Key Features

- Real-time HTTP message reception using `HttpListener`
- Events for connection, disconnection, errors, and new message dispatch
- Structured message model including timestamps, URL segments, and raw/parsed payload
- Fully documented C# code using .NET 8

Architecture Overview

The DAS acts as a lightweight server that receives messages from a test cell. Each message is parsed, validated, and dispatched to an event system.

Project Structure Diagram



Example: Getting Started

Here's a quick example of how to instantiate and use the `DASMessageListener` class:

```
using Teradyne.Archimedes.AMPCommunication;
using Teradyne.Archimedes.DAS.Listener;

public class Program
{
    public static async Task Main(string[] args)
    {
        Console.WriteLine("Hello, Welcome to my DAS!");

        var listener = new DASMessageListener("http://localhost:3000/tems/");

        listener.Connected += () => Console.WriteLine("Connected to DAS.");
        listener.Disconnected += () => Console.WriteLine("Disconnected.");
        listener.Error += (ex) => Console.WriteLine($"Error: {ex.Message}");
        listener.NewMessage += (cellid, message) =>
        {
            Console.WriteLine($"New message from {cellid}: {message.MessageName}");
        };

        await listener.Start();
    }
}
```

This documentation is part of the Teradyne Archimedes Reference Architecture series.

Namespace Teradyne.Archimedes.AMPCommunication

Classes

AMPEvents

Provides utility definitions for the DAS system, including event delegate declarations.

Delegates

AMPEvents.NewMessageDlg

Delegate for handling new message events. Triggered when a new AMPMessageModel is received from a test cell.

Class AMPEvents

Namespace: Teradyne.Archimedes.AMPCommunication

Assembly: Teradyne.Archimedes.DAS.Listener.dll

Provides utility definitions for the DAS system, including event delegate declarations.

```
public class AMPEvents
```

Inheritance

object ← AMPEvents

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Delegate AMPEvents.NewMessageDlg

Namespace: Teradyne.Archimedes.AMPCommunication

Assembly: Teradyne.Archimedes.DAS.Listener.dll

Delegate for handling new message events. Triggered when a new AMPMessageModel is received from a test cell.

```
public delegate void AMPEvents.NewMessageDlg(string cellid, AMPMessageModel message)
```

Parameters

`cellid` string

The identifier of the test cell that received the message.

`message` AMPMessageModel

The AMP message payload.

Namespace Teradyne.Archimedes.AMPCommunication.Messages

Classes

AMPMessageManager

Manages AMP message subscriptions and interactions. It is mandatory to return a list of message names as a response to the INITIALIZATION message and recommended for each message.

Class AMPMessageManager

Namespace: Teradyne.Archimedes.AMPCommunication.Messages

Assembly: Teradyne.Archimedes.DAS.Listener.dll

Manages AMP message subscriptions and interactions. It is mandatory to return a list of message names as a response to the INITIALIZATION message and recommended for each message.

```
public class AMPMessageManager
```

Inheritance

object ← AMPMessageManager

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

MessageList

Current list of subscribed messages.

```
public HashSet<string> MessageList { get; }
```

Property Value

HashSet<string>

Methods

ExtractMessageListFromInitMessage(AMPMessageModel)

Extracts the message list from an INITIALIZATION message.

```
public void ExtractMessageListFromInitMessage(AMPMessageModel message)
```

Parameters

`message` AMPMessageModel

The INITIALIZATION message model.

Subscribe(IEnumerable<string>)

Subscribes to multiple message names.

```
public void Subscribe(IEnumerable<string> messageNames)
```

Parameters

`messageNames` IEnumerable<string>

A collection of message names to subscribe to.

Subscribe(string)

Subscribes to a single message name if it is valid.

```
public bool Subscribe(string messageName)
```

Parameters

`messageName` string

The name of the message to subscribe to.

Returns

bool

True if subscribed, false if already present or invalid.

SubscribeAll()

Subscribes to all predefined AMP messages.

```
public void SubscribeAll()
```

ToJson()

Serializes the message list into a JSON string.

```
public string ToJson()
```

Returns

string

A JSON string containing the MESSAGE_LIST.

UnsubscribeAll()

Clears the current message subscription list.

```
public void UnsubscribeAll()
```

UpdateListToMatchAvailableMessages(AMPMessageManager)

Intersects the message list with another manager's list. Only shared messages are kept.

```
public void UpdateListToMatchAvailableMessages(AMPMessageManager available)
```

Parameters

`available` AMPMessageManager

Manager with the available message list.

Namespace Teradyne.Archimedes.AMPCommunication.Models

Classes

AMPMessageModel

Represents an AMP message with metadata, test cell ID, timestamps, and JSON message formatting.

Class AMPMessageModel

Namespace: Teradyne.Archimedes.AMPCommunication.Models

Assembly: Teradyne.Archimedes.DAS.Listener.dll

Represents an AMP message with metadata, test cell ID, timestamps, and JSON message formatting.

```
public class AMPMessageModel
```

Inheritance

object ← AMPMessageModel

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Properties

FormattedMessage

The JSON-formatted version of the message. Falls back to raw message if invalid.

```
public string FormattedMessage { get; }
```

Property Value

string

IsInitialization

Determines whether the message name indicates an initialization command.

```
public bool IsInitialization { get; }
```

Property Value

bool

Message

The raw JSON message string. When set, attempts to format it with indentation.

```
public string Message { get; set; }
```

Property Value

string

MessageName

The name of the message extracted from the URL.

```
public string MessageName { get; set; }
```

Property Value

string

TestCellID

The identifier of the test cell associated with the message.

```
public string TestCellID { get; set; }
```

Property Value

string

TimeStamp

The timestamp of the message, either parsed or defaulting to UTC now.

```
public DateTime TimeStamp { get; set; }
```

Property Value

DateTime

URL

The URL segments associated with the message.

```
public string[] URL { get; set; }
```

Property Value

string[]

Namespace Teradyne.Archimedes.DAS.Listener

Classes

DASMessageListener

DASMessageListener is responsible for starting and stopping the DAS HTTP listener, receiving incoming HTTP requests, and dispatching them to a processor. It manages event notifications for connection, disconnection, errors, and new messages.

Delegates

DASMessageListener.DasErrorHandler

Delegate when the DAS is facing to an error

DASMessageListener.DasEventHandler

Delegate used for the event related to the DAS

Class DASMessageListener

Namespace: Teradyne.Archimedes.DAS.Listener

Assembly: Teradyne.Archimedes.DAS.Listener.dll

DASMessageListener is responsible for starting and stopping the DAS HTTP listener, receiving incoming HTTP requests, and dispatching them to a processor. It manages event notifications for connection, disconnection, errors, and new messages.

```
public class DASMessageListener : IDisposable
```

Inheritance

object ← DASMessageListener

Implements

IDisposable

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

DASMessageListener(string)

Initializes a new instance of the DASMessageListener class with a DAS URL.

```
public DASMessageListener(string dasurl)
```

Parameters

`dasurl` string

The URL to listen to.

Exceptions

ArgumentNullException

Fields

_messageList

System for managing the list of messages we want to receive

```
protected AMPMessageManager _messageList
```

Field Value

AMPMessageManager

Properties

DASURL

Gets the DAS listener URL.

```
public string DASURL { get; }
```

Property Value

string

Methods

Dispose()

Disposes the listener and stops it if necessary.

```
public void Dispose()
```

Start()

Starts the HTTP listener and begins processing incoming requests.

```
public Task<bool> Start()
```

Returns

Task <bool>

True if started successfully, false otherwise.

Stop()

Stops the HTTP listener and cancels all ongoing tasks.

```
public void Stop()
```

SubscribeToMessages(List<string>, bool)

Subscribes to a set of message names, optionally resetting previous subscriptions.

```
public void SubscribeToMessages(List<string> messageNames, bool resetFirst = true)
```

Parameters

`messageNames` List <string>

List of message names to subscribe to.

`resetFirst` bool

Whether to clear existing subscriptions first.

Events

Connected

Event when the DAS is connected

```
public event DASMessageListener.DasEventHandler Connected
```

Event Type

DASMessageListener.DasEventHandler

Disconnected

Event when the DAS is disconnected

```
public event DASMessageListener.DasEventHandler Disconnected
```

Event Type

DASMessageListener.DasEventHandler

Error

Event when the DAS encouterred and error

```
public event DASMessageListener.DasErrorHandler Error
```

Event Type

DASMessageListener.DasErrorHandler

NewMessage

Event when the DAS received a new message

```
public event AMPEvents.NewMessageDlg NewMessage
```

Event Type

AMPEvents.NewMessageDlg

Delegate DASMessageListener.DasErrorHandler

Namespace: Teradyne.Archimedes.DAS.Listener

Assembly: Teradyne.Archimedes.DAS.Listener.dll

Delegate when the DAS is facing to an error

```
public delegate void DASMessageListener.DasErrorHandler(Exception exception)
```

Parameters

`exception` Exception

Delegate DASMessageListener.DasEventHandler

Namespace: Teradyne.Archimedes.DAS.Listener

Assembly: Teradyne.Archimedes.DAS.Listener.dll

Delegate used for the event related to the DAS

```
public delegate void DASMessageListener.DasEventHandler()
```

📄 **Description**	🔗 **Download Link**
Entire Solution	The solution
GIT Hub Link	Git Hub

More details about those source code here

This documentation is part of the Teradyne Archimedes Reference Architecture series.

How to implement a DAS in Python for UE

Reference : RA 466

Description

This Reference Architecture describes how to create an AMP DAS in Python for the UltraEdge.

General Technical Info

Component	Version
Language	Python
Framework	3.11
Environment	Any
IGXL	10.60.02

Overview

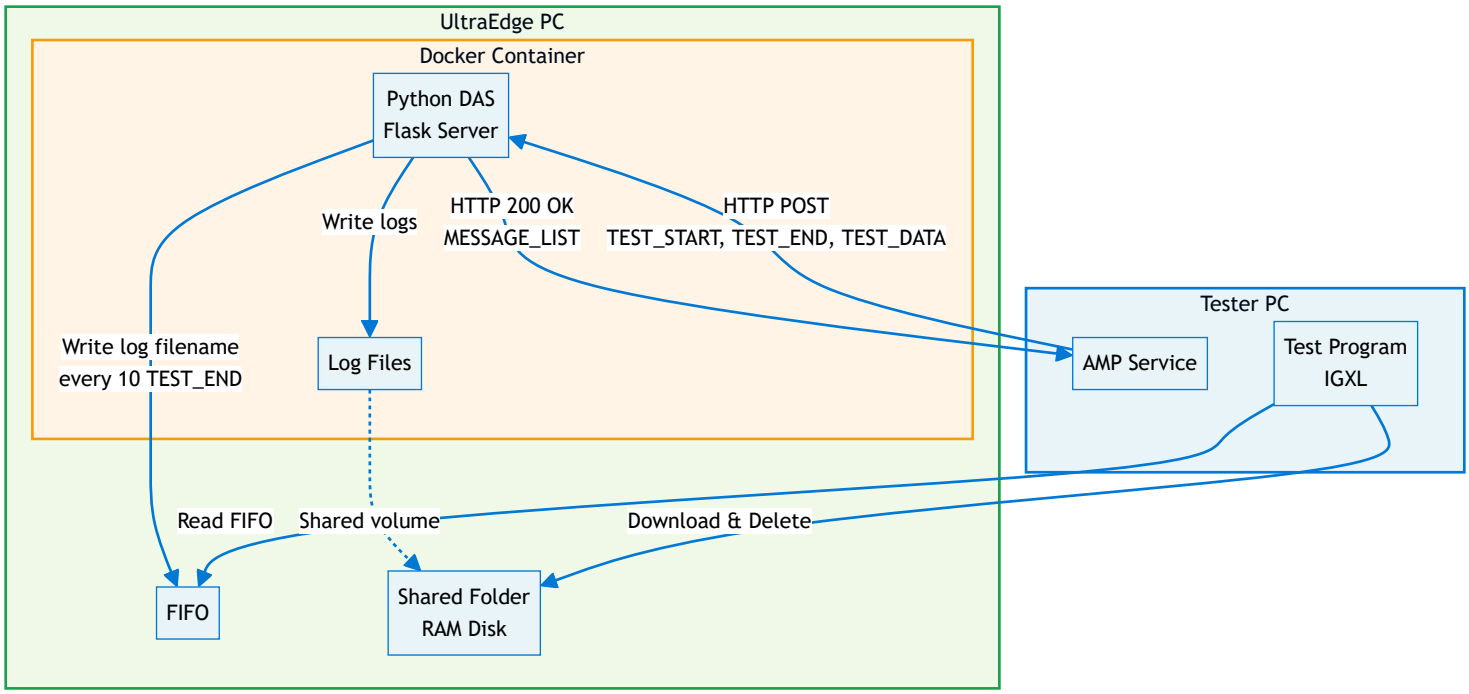
Creating an AMP DAS for the UltraEdge is simple but requires a few steps.
First of all, an AMP DAS is an HTTP server. Using Python, there exists many solutions to create an HTTP server.
The standard solution is to use `httplib2`. This package is already available on the UltraEdge.
Another interesting solution is to use Flask.
Of course, any HTTP server of your choosing can be used.

In this reference architecture, we show how:

- to create an AMP DAS using Flask
- to bundle it in a Docker

- to read data from a FIFO
- to read files from the UltraEdge even if they are generated in the Docker

Architecture Overview



Sample application

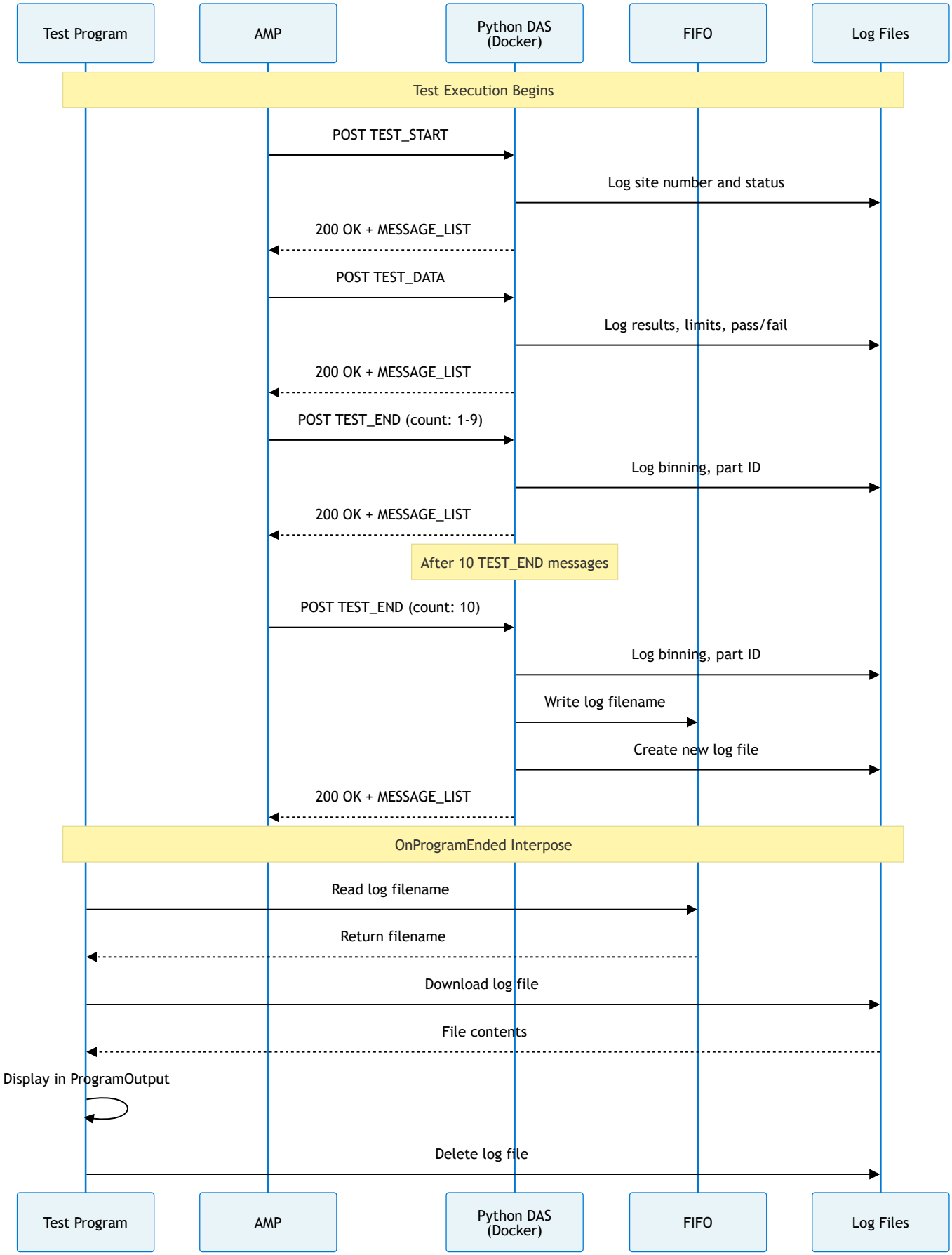
For this reference architecture, we create a DAS in Python running on the UltraEdge. This DAS logs the following information:

- `TEST_START`: site number and starting status
- `TEST_END`: site number, binning, pass/fail, part id
- `TEST_DATA`: results, limits, pass/fail

The log filename is computed using a simple touchdowns counter.
In addition, every 10 TEST_END message, the current log filename is sent to a FIFO and a new log filename is computed and used.

On the tester host, the test program will attempt to **read the FIFO** at each end of test (in the `OnProgramEnded` interpose function).
If a log filename can be read, the test program will **download** this file from the UltraEdge PC to display its contents in the ProgramOutput. It will also remotely **delete** this log file.

Data Flow Sequence



Creating an AMP DAS with Flask

`Flask` defines a simple HTTP server and uses Python decorators to handle HTTP messages.

> Note that `Flask` alone is **not** recommended for production.
> However, it can be wrapped within tools such as `Waitress` to provide a robust solution.
> See https://flask.palletsprojects.com/en/stable/deploying/ for more information.

An AMP URL is structured as follows:
`http://{server}:{port}/{DAS name}/{parameters}`

- `server`, `port`, and `DAS name` correspond to the DAS definition in the `AMP` configuration on the tester. See more details in the next chapter.
- The contents of `parameters` will vary according to the AMP message.

Example

`http://10.100.100.1:5001/PyDAS/TEST_CELL/UFLEXPLUS1/LOT/LOT123/SUBLOT/42/TEST_START`

Here, `http://10.100.100.1:5001/PyDAS/` is the DAS defined in AMP.
In addition, we can see 4 parameters: TEST_CELL, LOT, SUBLOT, message name. `TEST_CELL`, `LOT`, `SUBLOT` act as keywords in the URL and the value following the keywords correspond to that keyword.

- `UFLEXPLUS1` is the `TEST_CELL` hostname of the tester PC.
- `LOT123` is the current `LOT`
- `42` is the current `SUBLOT`
- `TEST_START` is the AMP message.

Setting up a DAS with the AMP software

To setup a DAS for AMP on the tester PC, the following command must be used in an Administrator command line: `ampshell -das add /n {DAS name} /u {DAS URL}`
For example: `ampshell -das add /n PyDAS /u http://10.100.100.1:5001/PyDAS/`

> Note that the `enable_das.bat` script is provided in the `source` folder to run this command for you.

Starting a Flask server

To run a Flask server, the following is required:

- Import Flask: `from flask import Flask`
- Create a Flask object: `app = Flask(__name__)`
- Start the server using a chosen port: `app.run(host='0.0.0.0', port=5001)`, the IP address 0.0.0.0 binds to the local host fo the UltraEdge and makes the server accessible from the tester PC.

Using the Flask `route` decorator

Flask defines a decorator `route` to simplify the management of specific URLs. In the case of AMP, this may come handy to manage each message.
For example, a function to manage TEST_DATA messages can be created and the Flask server automatically calls this function for each AMP TEST_DATA message received.\

> Note that the `POST` method must be used for all AMP messages.

```
@app.route('/PyDAS/TEST_CELL/UFLEXPLUS1/LOT/LOT123/SUBLOT/42/TEST_DATA', methods=['POST'])
def handle_TEST_DATA():
  ...
```

There is one obvious limitation to this solution: the code must be updated for each tester PC, each lot and each sublot.
Fortunately, Flask allows placeholders in the URL string that are mapped to a function parameter, so the code can be updated as follows:

```
@app.route('/PyDAS/TEST_CELL/<string:cellhost>/LOT/<string:lotid>/SUBLOT/<string:sublotid>/T
EST_DATA', methods=['POST'])
def handle_TEST_DATA(cellhost, lotid, sublotid):
  ...
```

Reusing our example, when the DAS receives the TEST_DATA message from AMP, the function `handle_TEST_DATA` is called with:

- `cellhost` = `UFLEXPLUS1`
- `lotid` = `LOT123`
- `sublotid` = `42`

Bundling the application in a Docker

Since the UltraEdge is not accessible at all, installing new applications is not possible. Fortunately, using Docker containers is the solution to implement applications as we see fit.

In the `source` folder, a `Dockerfile` and a script to create a Docker package are provided.
See also, the RA 448 on creating a Docker file.

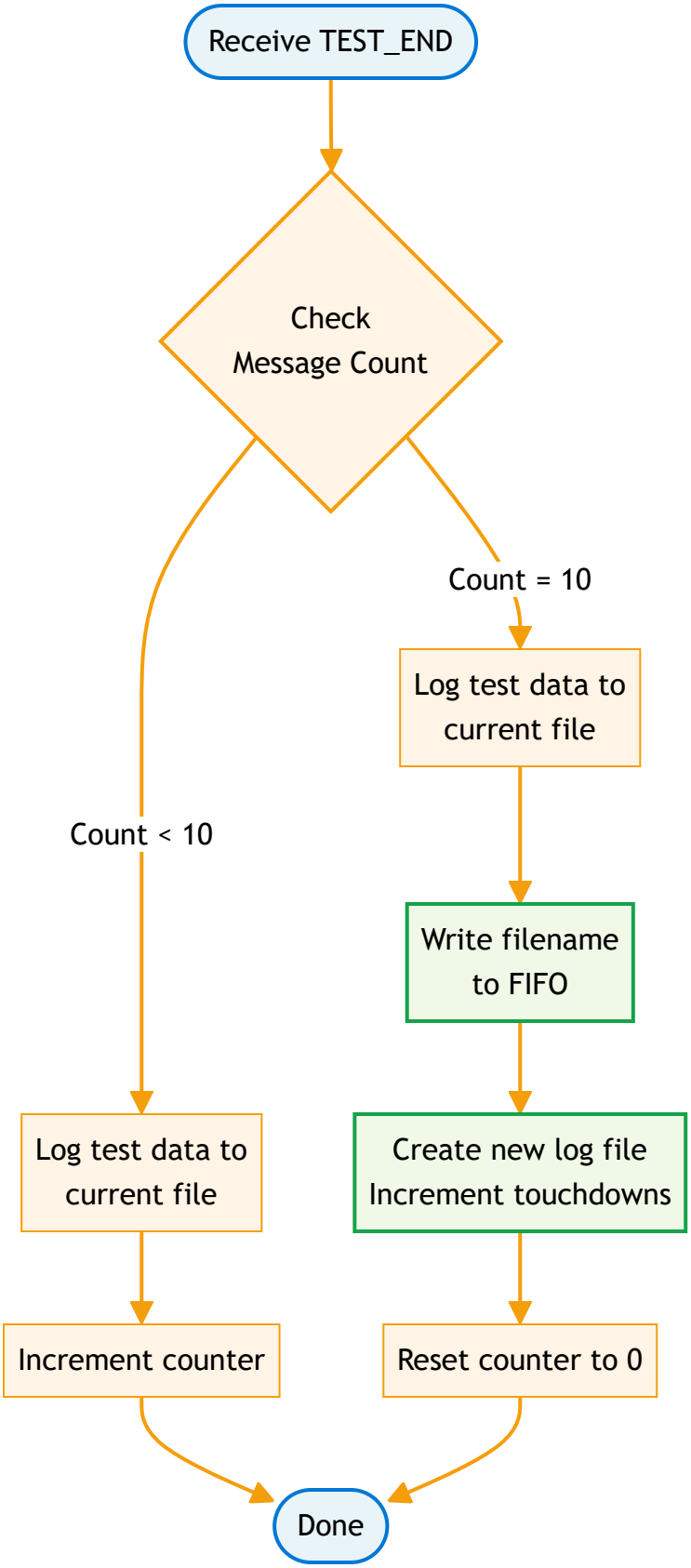
Reading data and file from the UltraEdge

The UltraEdge library available on the tester provides an API to read and write FIFOs defined on the UltraEdge PC.
In this example, the test program attempts to read the FIFO at each end of test and the AMP Python DAS

writes to this FIFO every 10 `TEST_END` message.
The information written to the FIFO is the name of a log file to copy from the UltraEdge to the tester PC.

There is one piece missing: the application runs inside a Docker container and the log file is written inside this container. How can we copy this file?
The UltraEdge software automatically creates a shared folder between the UltraEdge RAM disk filesystem and any Docker container. Any file written to the folder within the Docker container will be visible on the UltraEdge RAM disk folder in the current session.

Log File Management Logic



Test program

In this sample, the test program sends the application to the UltraEdge and accesses the data required to display information.

In the test program, a module was created to take care of UltraEdge communication.
An initialization function (called `UE_Start`) is called after validation and does the following:

1. Connects to the UltraEdge
2. Starts a session
3. Sets the FIFO up
4. Sends the Docker file to the UltraEdge
5. Tells the UltraEdge to load the Docker image
6. Tells the UltraEdge to run a Docker container

Upon exiting IGXL, a function called `UE_Stop` (called in `OnProgramClose`) performs cleanup operations:

1. Terminates the application
2. Closes the FIFO
3. Ends the session
4. Disconnect from the UltraEdge

An additional function `GetAndDisplayLogFile` is invoked from the `OnProgramEnded` interpose function.

1. Attempts to read data from the FIFO. If there is nothing to read, stop there.
2. If the FIFO contains a string, this is the filename to get from the UltraEdge.
3. Copy the file from the UltraEdge.
4. Delete the file from the UltraEdge.
5. Read the contents of the file.
6. Display the contents to the ProgramOutput.

Source code

The API reference for the Python code can be found here

Component	Comments
`pydas.py`	Python DAS running in a Docker on the UltraEdge
`uetrial.igxl`	Test Program
`SimulatedConfig.txt`	IGXL configuration file to simulate the test program channelmap
`Dockerfile`	Docker description file
`makedocker.sh`	Shell script to create the Docker package
`enable_das.bat`	Support script to add the DAS to AMP

Description	Download Link
`RA_0466.zip`	Download
GIT Hub Link	Git Hub

More details about those source code here

This documentation is part of the Teradyne Archimedes Reference Architecture series.

How to implement a high-performance DAS in C++23 using Boost

Reference : RA 617

Welcome to the documentation for the **RA_0617 BoostDAS**.
This reference architecture demonstrates how to build a high-performance Data Analytics Solution (DAS) using modern C++23 and the Boost libraries, with cross-platform compilation support via the Zig build system.

General Technical Info

Component	**Version**
Language	C++23
Libraries	Boost (Beast, ASIO, JSON)
Build System	Zig 0.15.2
Environment	Windows 10/11, Linux
IGXL	> 10.30.10
MST	> 2016C
IDE	Visual Studio Code

Key Features

- **High Performance**: Native C++ implementation with optimized builds as small as 600KB
- **Cross-Platform**: Compile for Windows and Linux from any development machine
- **HTTP Listener**: Real-time message reception using Boost.Beast

- **Configurable Logging**: Enable or disable logging at compile-time for optimal performance
- **Flexible Deployment**: Run locally or deploy to UltraEdge without .NET runtime
- **Command-Line Configuration**: Customize DAS URL and log file path via runtime arguments

Architecture Overview

The BoostDAS is a lightweight HTTP server built with Boost.Beast that receives messages from test programs. It features compile-time configurability for logging and listener components, allowing you to optimize the binary for specific deployment scenarios.

Performance Characteristics

- **Statically Linked Build**: ~600KB with all logging disabled
- **Debug Build**: ~5.5MB with full symbols
- **Optimized Release**: ~740KB with size optimization
- **No Runtime Dependencies**: Fully standalone executable on Linux (musl build)

Example: Quick Start

Building the DAS

To build for your local environment:

```
zig build
```

To build an optimized version for deployment:

```
zig build -Doptimize=ReleaseSmall
```

To build for Linux UltraEdge (from Windows):

```
zig build -Dtarget=x86_64-linux-musl -Doptimize=ReleaseSmall
```

Running the DAS

Run with default settings (port 4242, path "/cppdas/"):

```
zig build run
```

Run with custom configuration:

```
zig build run -- -dasUrl "http://localhost:8080/mydas/" -dasLog mylog.txt
```

Run the compiled executable directly:

```
./zig-out/x86_64-windows-gnu/Debug/cppdas.exe
```

To stop the DAS, type 'Q' in the terminal.

Build Options

Compile-Time Flags

Disable logging for maximum performance:

```
zig build -Dnologs
```

Disable HTTP listener at compile-time:

```
zig build -Dnolistener
```

Cross-Compilation Targets

```
# Windows (default on Windows hosts)
zig build -Dtarget=x86_64-windows-gnu

# Linux with dynamic linking
zig build -Dtarget=x86_64-linux-gnu

# Linux with static linking (standalone)
zig build -Dtarget=x86_64-linux-musl
```

Runtime Options

- `-dasLog [file_name]`: Specify log file path (default: "DasLog.log")
- `-dasUrl [url]`: Specify DAS URL with port and path (default: "http://localhost:4242/cppdas/")
- `-h` or `--help`: Display help information

Documentation

Build and serve the full API documentation:

```
zig build docs
```

Then open your browser to the URL displayed in the console.

This documentation is part of the Teradyne Archimedes Reference Architecture series.

What is Zig? A Beginner's Guide

Introduction

Zig is a modern programming language and build system designed to replace C/C++ in systems programming. Think of it as a "better C" that fixes many of the problems developers face when writing low-level code.

While this project uses **C++23** for the actual application code, we use **Zig as a build system** instead of traditional tools like CMake or Make.

What Does Zig Do in This Project?

In our `boostdas` project:

- **We write code in C++23** (in `src/main.cpp` and headers)
- **Zig compiles our C++ code** (acts as a smart compiler wrapper)
- **Zig manages dependencies** (downloads and links Boost libraries automatically)
- **Zig produces the final executable** (`cppdas.exe`)

Think of Zig as a **super-smart project manager** that handles all the complicated build steps for us.

Zig Build Process

```
Download Zig → Run: zig build → Success ✓
```

Key Benefits for Beginners

1. Zero Dependencies

- Download one `.zip` file (88 MB for Windows)
- Extract it
- You're done! No installers, no system configuration

Example:

```
# Traditional C++ setup
choco install cmake
choco install llvm
vcpkg install boost
# ... 2 hours later ...

# Zig setup
Invoke-WebRequest -Uri "https://ziglang.org/download/0.15.2/zig-x86_64-windows-0.15.2.zip"
Expand-Archive zig-x86_64-windows-0.15.2.zip
# Done in 5 minutes!
```

2. **Automatic Dependency Management**

Zig downloads and manages libraries automatically through `build.zig.zon`:

```
.dependencies = .{
    .boost = .{
        .url = "https://boostorg.jfrog.io/artifactory/...",
        .hash = "...",
    },
}
```

You never need to:

- Manually download Boost
- Set up environment variables
- Configure include paths
- Link libraries manually

3. **Cross-Compilation Made Easy**

To build for Linux while on Windows, specify the target flag:

```
# Build for Windows (default)
zig build -Doptimize=ReleaseSmall

# Build for Linux from Windows
zig build -Dtarget=x86_64-linux-gnu -Doptimize=ReleaseSmall

# Build for macOS from Windows
zig build -Dtarget=x86_64-macos -Doptimize=ReleaseSmall
```

No need for virtual machines or separate build environments!

4. Simple Build Configuration

Compare these two files that do the same thing:

CMakeLists.txt (Traditional)

```
cmake_minimum_required(VERSION 3.20)
project(cppdas CXX)

set(CMAKE_CXX_STANDARD 23)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(Boost REQUIRED COMPONENTS system filesystem json)

add_executable(cppdas src/main.cpp)
target_include_directories(cppdas PRIVATE src/include)
target_link_libraries(cppdas PRIVATE Boost::system Boost::filesystem)
# ... 50+ more lines of configuration ...
```

build.zig (Zig)

```
const exe = b.addExecutable(.{
    .name = "cppdas",
    .root_source_file = "src/main.cpp",
    .target = target,
    .optimize = optimize,
});

exe.linkSystemLibrary("c++");
exe.addIncludePath("src/include");
b.installArtifact(exe);
```

Clean, simple, readable!

5. Built-in Optimization Levels

Zig provides four optimization modes out of the box:

```
# Debug - Fast compilation, large binary, debug symbols
zig build

# ReleaseFast - Maximum speed
zig build -Doptimize=ReleaseFast

# ReleaseSmall - Minimum binary size (what we use!)
zig build -Doptimize=ReleaseSmall
```

```
# ReleaseSafe - Fast + safety checks
zig build -Doptimize=ReleaseSafe
```

Real results from our project:

- Debug: 5.5 MB
- ReleaseSmall: 535 KB (10x smaller!)

6. **Reproducible Builds**

The same `build.zig` file produces the **exact same binary** on:

- Windows 10, 11
- Linux (Ubuntu, Fedora, Arch)
- macOS (Intel and Apple Silicon)

No more "it works on my machine" problems!

Real-World Example: Our Project Journey

Before Zig (Hypothetical CMake Setup)

1. Install Visual Studio Build Tools (6 GB)
2. Install CMake (100 MB)
3. Install vcpkg (package manager)
4. Download Boost (500 MB)
5. Configure paths for 30 minutes
6. Debug linker errors for 2 hours
7. Finally compile

Total time: 3-4 hours
Disk space: ~7 GB

With Zig (What We Actually Did)

1. Download Zig (88 MB)
2. Run `zig build -Doptimize=ReleaseSmall`
3. Get `cppdas.exe` (535 KB)

Total time: 10 minutes
Disk space: 88 MB

Common Questions

"Is Zig only for Zig code?"

No! Zig can compile:

- C code
- C++ code (like our project)
- Zig code
- Mix of all three

"Will I need to learn Zig language?"

Not necessarily! You only need to understand `build.zig` configuration, which is simpler than CMake. You can keep writing C++ as usual.

"Is it production-ready?"

The build system: YES. Many companies use Zig to compile C/C++ projects in production.

The language itself: Still evolving (currently version 0.15.2), but the build system is stable.

"What if I get errors?"

Zig provides **excellent error messages**:

```
error: unable to find 'boost/asio.hpp'
note: add this path to build.zig:
    exe.addIncludePath("path/to/boost/include");
```

Compare to CMake:

```
CMake Error at CMakeLists.txt:15 (find_package):
  Could not find a configuration file for package "Boost"
  # ... 200 lines of confusing output ...
```

Zig Build System Architecture

Here's how Zig manages our C++ project:

```
blockdiag {
  orientation = portrait;

  A [label = "build.zig.zon\n(Dependencies)", color = "#E1F5FF"];
  B [label = "Download Boost\nLibraries", color = "#FFF4E1"];
  C [label = "build.zig\n(Build Config)", color = "#E1F5FF"];
  D [label = "Compile C++\nwith Flags", color = "#FFE8E8"];
  E [label = "Link All\nLibraries", color = "#FFE8E8"];
  F [label = "cppdas.exe\n(Executable)", color = "#E8FFE8"];
```

```
  G [label = ".zig-cache/\n(Fast Rebuilds)", color = "#F0F0F0", shape = "flowchart.database"];

  A -> B -> C -> D -> E -> F;
  B -> G;
  D -> G;
  E -> G;
}
```

Everything is **cached** in `.zig-cache/`, so rebuilds are fast!

Performance Comparison

Build System	First Build	Rebuild	Binary Size	Setup Time
CMake + MSVC	45 seconds	12 sec	1.2 MB	3-4 hours
Zig	30 seconds	8 sec	535 KB	10 minutes

Conclusion

Zig is NOT just another programming language. It's a complete build toolchain that makes C/C++ development:

- ✅ **Easier** - No complex setup
- ✅ **Faster** - Optimized builds
- ✅ **Portable** - Works everywhere
- ✅ **Reproducible** - Same code = same binary

For our `boostdas` project, Zig allowed us to:

1. Compile C++23 code without installing Visual Studio
2. Automatically manage Boost dependencies
3. Produce a tiny 535 KB executable
4. Share the project with zero setup instructions

Think of Zig as the "npm" or "cargo" for C/C++ - a modern package manager and build system that finally brings C++ into the 21st century!

Docker Integration

Zig's cross-compilation makes it **perfect for Docker**:

Traditional C++ Docker Image

```
FROM gcc:latest  # 1.2 GB base image
COPY . .
RUN g++ ... # Complex build commands
# Final image: ~1.3 GB
```

Zig + Docker Multi-Stage Build

```
FROM alpine:latest AS builder  # 7 MB base
RUN wget zig && tar -xf zig
COPY . .
RUN zig build  # One simple command

FROM alpine:latest  # 7 MB runtime
COPY --from=builder /build/cppdas /app/
# Final image: ~12 MB (99% smaller!)
```

Benefits:

- ✅ **99% smaller images** (12 MB vs 1.3 GB)
- ✅ **Faster deployments** (seconds instead of minutes)
- ✅ **Lower bandwidth costs** (crucial for CI/CD)
- ✅ **Cross-platform builds** (build ARM from x86, etc.)

See our `Dockerfile` and `DOCKER.md` for complete examples!

Resources

- **Official Website:** https://ziglang.org
- **Download Zig:** https://ziglang.org/download/
- **Our Installation Guide:** See `INSTALLATION.md` in this repository
- **Our Build Guide:** See `COMPILATION.md` in this repository
- **Our Docker Guide:** See `DOCKER.md` in this repository
- **Documentation:** https://ziglang.org/documentation/master/

Bottom Line: You don't need to learn Zig to benefit from it. Use it as your C++ build system and leverage its simplicity and cross-platform capabilities.

Zig Installation Guide

This guide explains how to install Zig 0.15.2 to build the boostdas project.

Prerequisites

- Windows 10/11 (x64)
- Internet connection
- PowerShell 5.1 or later

Required Version

This project requires **Zig version 0.15.2** exactly. Other versions may not work.

Method 1: Manual Download (Recommended)

Step 1: Download Zig

Option A: Manual download

1. Visit the official Zig downloads page:

   ```
   https://ziglang.org/download/
   ```

2. Download the Windows x64 version for 0.15.2:

   - **ZIP file (recommended)**: `zig-x86_64-windows-0.15.2.zip`
   - Direct URL: https://ziglang.org/download/0.15.2/zig-x86_64-windows-0.15.2.zip
   - Size: approximately 88 MB

Option B: Automatic download via PowerShell

Open PowerShell in the project folder and run:

```
$url = "https://ziglang.org/download/0.15.2/zig-x86_64-windows-0.15.2.zip"
$output = "$PWD\zig.zip"
Invoke-WebRequest -Uri $url -OutFile $output -UseBasicParsing
Write-Host "✓ Downloaded: $([math]::Round((Get-Item $output).Length / 1MB, 2)) MB"
Expand-Archive -Path $output -DestinationPath $PWD -Force
Remove-Item $output
Write-Host "✓ Extraction complete"
```

Step 2: Extract the Archive

The extraction will create a folder named `zig-x86_64-windows-0.15.2`

Recommended locations:

- **Option A**: In the project folder
  `C:\Users\bonneval\source\repos\boostdas\zig-x86_64-windows-0.15.2\`
- **Option B**: System-wide location
  `C:\Program Files\zig-x86_64-windows-0.15.2\`

Step 3: Add Zig to PATH

Temporary Method (current PowerShell session only):

If Zig is in the project folder:

```
$env:PATH = "$PWD\zig-x86_64-windows-0.15.2;$env:PATH"
```

For another location:

```
$env:PATH = "C:\path\to\zig-x86_64-windows-0.15.2;$env:PATH"
```

Permanent Method (recommended):

1. Open System Properties:
   - Right-click "This PC" → Properties
   - Advanced system settings
   - Environment Variables
2. Edit the `Path` variable:
   - Under "System variables" or "User variables"
   - Click "Edit"

    - Add the path: `C:\Program Files\zig-x86_64-windows-0.15.2` (or your chosen location)
    - Click OK
3. **Important**: Restart PowerShell to apply changes

Step 4: Verify Installation

Open a new PowerShell terminal and run:

```
zig version
```

Expected output:

```
0.15.2
```

If the command fails with "zig: The term 'zig' is not recognized", verify that:

- The PATH was correctly configured
- You restarted PowerShell after modifying the PATH

Method 2: Local Usage (without PATH)

If you prefer not to modify the PATH, you can use Zig directly:

```
# Check version
.\zig-x86_64-windows-0.15.2\zig.exe version

# Build the project
.\zig-x86_64-windows-0.15.2\zig.exe build
```

Method 3: Package Managers (Alternative)

⚠️ **Warning**: Package managers may not have the exact version 0.15.2.

Chocolatey (if installed):

```
choco install zig --version=0.15.2
```

Scoop (if installed):

```
scoop install zig@0.15.2
```

After installation via package manager, verify the version:

```
zig version
```

Troubleshooting

Issue: "zig: The term 'zig' is not recognized"

Cause: Zig is not in the PATH.

Solution:

- Verify you added the correct path to PATH
- Restart PowerShell after modifying PATH
- Or use the full path: `C:\path\to\zig\zig.exe version`

Issue: Wrong version displayed

Cause: Another version of Zig is installed on the system.

Solution:

- Check the order of paths in PATH
- Place the path to Zig 0.15.2 first
- Or uninstall other Zig versions

Issue: Download fails or 0 byte file

Cause: Incorrect URL or version unavailable.

Solution:

- **Correct URL**: `https://ziglang.org/download/0.15.2/zig-x86_64-windows-0.15.2.zip`
- ⚠️ Note: The order is `zig-x86_64-windows-` not `zig-windows-x86_64-`
- Try downloading manually via browser if PowerShell fails
- Check your Internet connection and firewall

Alternative: Use the full PowerShell script in the "Step 1" section

Next Steps

Once Zig is installed and verified, consult the README.md for:

- Building the project
- Running the application

- Compilation options

Last Updated: February 4, 2026
Required Zig Version: 0.15.2
Tested On: Windows 11 x64

Compilation Guide

This guide covers building the boostdas project using the Zig build system.

Prerequisites

- Zig 0.15.2 installed (see INSTALLATION.md)
- Internet connection (for first build to fetch dependencies)
- Windows 10/11 with PowerShell

First-Time Setup

1. Create Required Directories

On first build, you may encounter a temporary directory error. Create it manually:

```
New-Item -ItemType Directory -Path "$env:LOCALAPPDATA\zig\tmp" -Force
```

2. Fetch Dependencies

The build system will automatically download Boost libraries on first build. Optionally, you can fetch dependencies explicitly:

```
zig build --fetch
```

Basic Build Commands

Default Debug Build

```
zig build
```

Output Location: `zig-out\x86_64-windows-gnu\Debug\cppdas.exe`
Size: ~5.5 MB (Debug build includes symbols)

Build and Run

```
zig build run
```

This builds the project and immediately runs the executable.

List Available Build Steps

```
zig build --list-steps
```

Available steps:

- `install` (default) - Build and copy artifacts
- `run` - Build and run the application
- `docs` - Build Doxygen documentation
- `uninstall` - Remove build artifacts

Optimization Levels

Use `-Doptimize` to control optimization:

ReleaseSmall (Smallest Binary)

```
zig build -Doptimize=ReleaseSmall
```

Output: `zig-out\x86_64-windows-gnu\ReleaseSmall\cppdas.exe`
Size: ~740 KB

ReleaseFast (Maximum Performance)

```
zig build -Doptimize=ReleaseFast
```

ReleaseSafe (Optimized with Safety Checks)

```
zig build -Doptimize=ReleaseSafe
```

Debug (Default, with Symbols)

```
zig build -Doptimize=Debug
```

Compile-Time Feature Flags

The project supports conditional compilation to exclude functionality:

Disable Logging

Removes all logging code at compile-time:

```
zig build -Dnologs
```

Benefits: Smaller binary, better performance

Generate Compilation Database

For IDE integration (code completion, navigation):

```
zig build -Dcompiledb
```

This creates `compile_commands.json` in the project root.

Combined Build Examples

Development Build with Compilation Database

```
zig build -Dcompiledb=true
```

Cross-Compilation

Linux (Dynamic Linking)

```
zig build -Dtarget=x86_64-linux-gnu -Doptimize=ReleaseFast
```

Output: `zig-out\x86_64-linux-gnu\ReleaseFast\cppdas`

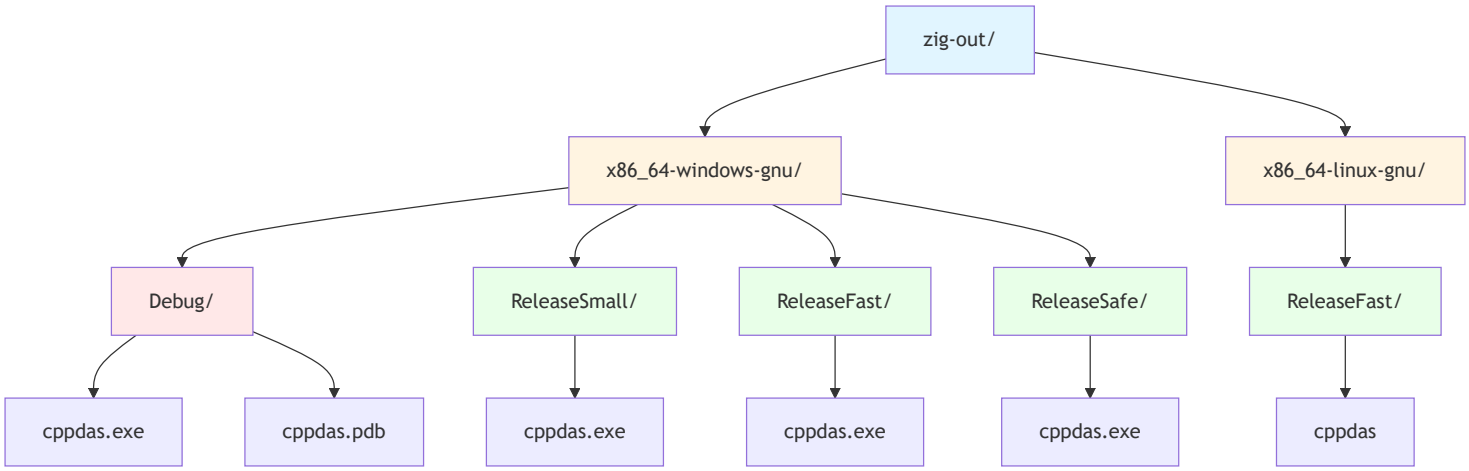
Linux (Static Linking with musl)

```
zig build -Dtarget=x86_64-linux-musl -Doptimize=ReleaseSmall
```

This creates a completely standalone binary with no external dependencies.

Build Output Structure

Build artifacts are organized by architecture, OS, ABI, and optimization level:



Running the Application

Run with Default Arguments

```
.\zig-out\x86_64-windows-gnu\Debug\cppdas.exe
```

Run with Custom Log Paths

```
zig build run -- -dasLog myDas.log -rebinLog myRebin.log
```

Note: The `--` separates build arguments from runtime arguments.

Show Runtime Help

```
zig build run -- -h
```

Or directly:

```
.\zig-out\x86_64-windows-gnu\Debug\cppdas.exe -h
```

Output:

```
Usage: cppdas [-dasLog [file_name]] [-rebinLog [file_name]]
```

Cleaning Build Artifacts

Remove Build Outputs

```
Remove-Item -Recurse -Force zig-out
```

Remove Cache (Forces Complete Rebuild)

```
Remove-Item -Recurse -Force zig-cache
Remove-Item -Recurse -Force .zig-cache
```

Clean All Build Artifacts

```
Remove-Item -Recurse -Force zig-out, zig-cache, .zig-cache
```

Advanced Build Options

Verbose Build Output

See all commands being executed:

```
zig build --verbose
```

Parallel Build Jobs

Control CPU usage (default uses all cores):

```
zig build -j4
```

Build Summary

Control what's printed:

```
zig build --summary all      # Show everything
zig build --summary failures # Show only failures (default)
zig build --summary none     # No summary
```

Troubleshooting

Issue: "unable to walk temporary directory"

Cause: Zig temp directory doesn't exist.

Solution:

```
New-Item -ItemType Directory -Path "$env:LOCALAPPDATA\zig\tmp" -Force
```

Issue: Build fails with Boost errors on first build

Cause: Network issues downloading dependencies.

Solution:

- Check Internet connection
- Try explicit fetch: `zig build --fetch`
- Check firewall settings

Issue: "FileNotFound" errors during build

Cause: Incomplete dependency fetch.

Solution:

```
# Clean and rebuild
Remove-Item -Recurse -Force .zig-cache, zig-cache
zig build --fetch
zig build
```

Issue: Binary size larger than expected

Cause: Debug build or features not disabled.

Solution:

```
# Build smallest possible binary
zig build -Doptimize=ReleaseSmall -Dnologs=true
```

Issue: Linker errors on Windows

Cause: Missing system libraries.

Solution: The build automatically links `Ws2_32` on Windows. If errors persist:

- Ensure you're using the correct target for your system
- Try rebuilding from clean state

Build Performance Tips

1. **Use incremental builds**: Zig caches build artifacts automatically
2. **First build is slow**: Dependencies are downloaded and compiled
3. **Subsequent builds are fast**: Only changed files are recompiled
4. **Parallel compilation**: Zig uses all CPU cores by default
5. **Cross-compilation is fast**: No additional toolchains needed

IDE Integration

Generate compile_commands.json

```
zig build -Dcompiledb=true
```

This enables:

- Code completion in VS Code (with C/C++ extension)
- Jump to definition
- Error checking in IDE
- Symbol navigation

Recommended VS Code Extensions

- C/C++ (Microsoft)
- clangd (alternative to C/C++)
- Zig Language (for build.zig syntax highlighting)

Binary Size Comparison

Configuration	Size	Notes
Debug (default)	~5.5 MB	Includes debug symbols (.pdb)
ReleaseSmall	~740 KB	Optimized for size
ReleaseSmall + -Dnologs	~700 KB	Further optimized
ReleaseFast	~800 KB	Optimized for speed

Next Steps

- See README.md for application usage
- See INSTALLATION.md for Zig installation
- Run `zig build -h` for all build options
- Run `zig build run -- -h` for runtime options

Last Updated: February 4, 2026
Zig Version: 0.15.2
Tested On: Windows 11 x64

Building a Data Acquisition System (DAS) with C++23 and Zig

Table of Contents

Project Overview

This guide shows how to build a **production-ready HTTP Data Acquisition System** using:

- **C++23** for modern, safe code
- **Boost.Asio** for async I/O
- **Boost.Beast** for HTTP protocol
- **Zig** as the build system and package manager

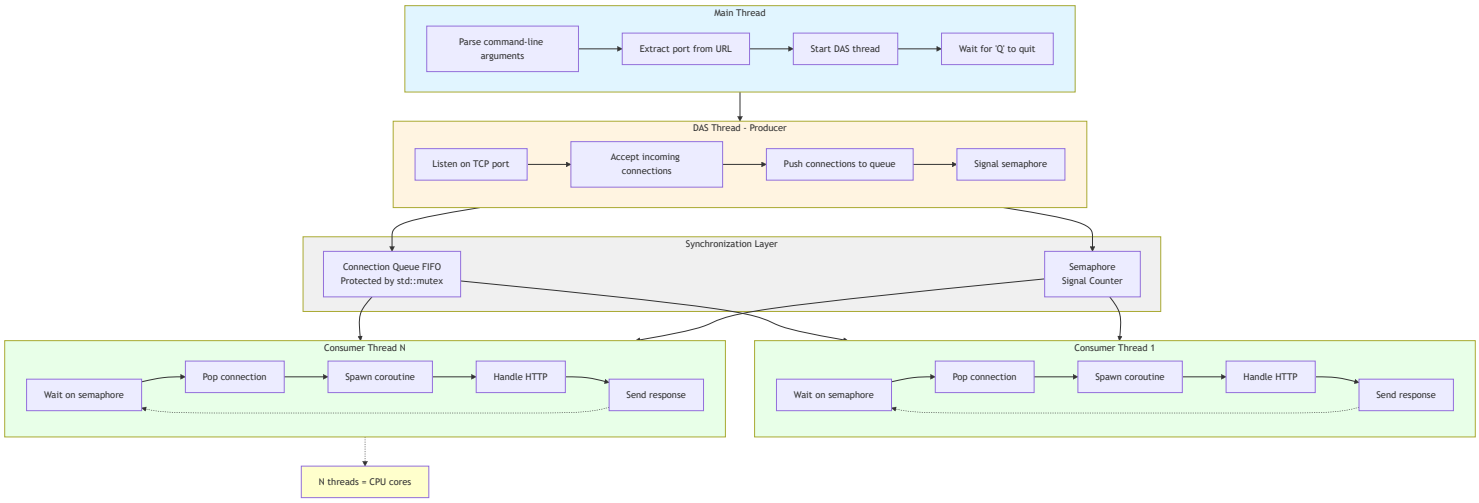
What the DAS does:

- Listens for HTTP POST requests on a configurable URL
- Validates that the first message is an INITIALIZATION message
- Logs all messages to a file
- Uses a multi-threaded producer-consumer pattern for high performance
- Returns HTTP 200 for valid requests, HTTP 500 for invalid first message

Final binary size: 535 KB
Performance: Handles thousands of concurrent connections

Architecture

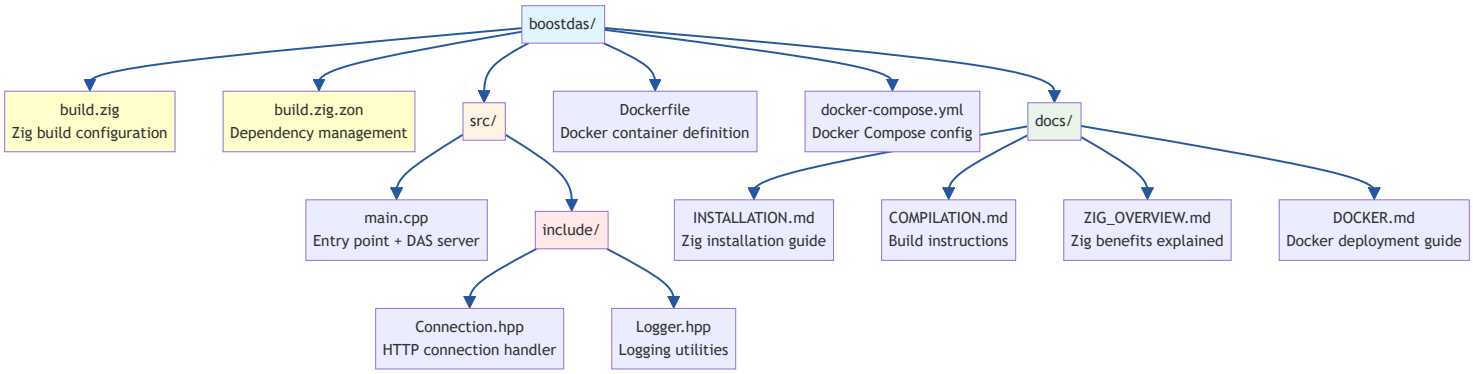
High-Level Design



Key Design Patterns

1. **Producer-Consumer Pattern**
    - Main thread produces connections
    - Worker threads consume and process them
    - Queue + semaphore for synchronization
2. **Coroutines (C++20/23)**
    - Non-blocking I/O operations
    - Better scalability than traditional threads
3. **RAII (Resource Acquisition Is Initialization)**
    - Automatic cleanup with destructors
    - No manual resource management
4. **Compile-Time Configuration**
    - Feature flags (`-Dnologs`, `-Dnolistener`)
    - Zero runtime overhead for disabled features

Project Structure



Setting Up the Build System

Step 1: Create `build.zig.zon` (Dependency File)

This file tells Zig what libraries to download:

```
.{
    .name = "cppdas",
    .version = "1.0.0",
    .dependencies = .{
        // Boost libraries - automatically downloaded and cached
        .@"boost.asio" = .{
            .url = "https://github.com/boostorg/asio/archive/refs/tags/boost-1.86.0.tar.gz",
            .hash = "1220...",  // Verification hash
        },
        .@"boost.beast" = .{
            .url = "https://github.com/boostorg/beast/archive/refs/tags/boost-1.86.0.tar.gz",
            .hash = "1220...",
        },
        // ... more Boost modules
    },
}
```

What happens:

1. First build: Zig downloads all dependencies
2. Subsequent builds: Uses cached versions (in `~/.cache/zig`)
3. Same versions across all machines (reproducible builds)

Step 2: Create `build.zig` (Build Configuration)

```
const std = @import("std");

pub fn build(b: *std.Build) void {
    // 1. Get build options from command line
    const target = b.standardTargetOptions(.{});
```

```
const optimize = b.standardOptimizeOption(.{});

// 2. Create the executable
const exe = b.addExecutable(.{
    .name = "cppdas",
    .target = target,
    .optimize = optimize,
});

// 3. Add C++ source file
exe.addCSourceFile(.{
    .file = b.path("src/main.cpp"),
    .flags = &.{
        "-std=c++23",           // Use C++23 standard
        "-Wall",                // All warnings
        "-Wextra",              // Extra warnings
        "-Wpedantic",           // Strict standard compliance
        "-Werror",              // Warnings = errors
    },
});

// 4. Link C++ standard library
exe.linkSystemLibrary("c++");

// 5. Link Windows socket library (platform-specific)
if (target.result.os.tag == .windows) {
    exe.linkSystemLibrary("Ws2_32");
}

// 6. Add include directories
exe.addIncludePath(b.path("src/include"));

// 7. Add Boost dependencies
const boost = b.dependency("boost.asio", .{});
exe.addIncludePath(boost.path("include"));
// ... repeat for all Boost modules

// 8. Compile-time feature flags
if (b.option(bool, "nologs", "Disable logging") orelse false) {
    exe.defineCMacro("NOLOG", null);
}
if (b.option(bool, "nolistener", "Disable listener") orelse false) {
    exe.defineCMacro("NOLISTENER", null);
}

// 9. Install the executable
```

```
    b.installArtifact(exe);
}
```

Build Commands:

```
# Debug build
zig build

# Release build (smallest binary)
zig build -Doptimize=ReleaseSmall

# Build without logging
zig build -Doptimize=ReleaseSmall -Dnologs=true

# Cross-compile for Linux from Windows
zig build -Dtarget=x86_64-linux-gnu -Doptimize=ReleaseSmall
```

Core Components Explained

1. Logger Template (`Logger.hpp`)

```
template <bool Disabled>
class Logger {
    std::ofstream file_;
public:
    Logger(std::string_view path) {
        if constexpr (!Disabled) {
            file_.open(std::string(path), std::ios::app);
        }
    }

    template <typename T>
    Logger& operator<<(const T& value) {
        if constexpr (!Disabled) {
            file_ << value;
        }
        return *this;
    }
};
```

Key Features:

- Template parameter `Disabled` evaluated at compile-time

- When `Disabled=true`, all logging code is eliminated (zero overhead)
- Uses `if constexpr` for compile-time branching

Usage:

```
Logger<false> logger{"app.log"};  // Logging enabled
logger << "Message";               // Writes to file

Logger<true> logger{"app.log"};   // Logging disabled
logger << "Message";               // Compiled to nothing (zero cost)
```

2. Connection Handler (`Connection.hpp`)

```
template <bool UseLogger>
class Connection {
    // Member variables
    boost::beast::http::request<boost::beast::http::string_body> request_;
    boost::beast::flat_buffer http_buffer_;
    std::unique_ptr<tcp::socket> socket_;
    Logger<UseLogger>& log_file_;
    std::string_view expected_path_;

public:
    // Constructor
    explicit Connection(
        std::unique_ptr<tcp::socket>&& socket,
        Logger<UseLogger>& log_file,
        std::string_view expected_path
    ) noexcept;

    // Async HTTP handler (C++20 coroutine)
    boost::asio::awaitable<void> TalkToClientAsync() {
        bool keep_alive{request_.keep_alive()};

        while (keep_alive) {
            // 1. Read HTTP request asynchronously
            co_await boost::beast::http::async_read(
                *socket_, http_buffer_, request_,
                boost::asio::use_awaitable
            );

            // 2. Validate request
            auto status = boost::beast::http::status::ok;

            if (request_.method() != boost::beast::http::verb::post) {
```

```
                status = boost::beast::http::status::bad_request;
            }
            else if (!request_.target().starts_with(expected_path_)) {
                status = boost::beast::http::status::not_found;
            }
            else {
                // First message validation
                bool expected = false;
                if (first_message_received.compare_exchange_strong(expected, true)) {
                    if (request_.target().find("INITIALIZATION") ==
std::string_view::npos) {
                        status = boost::beast::http::status::internal_server_error;
                    }
                }
            }

            // 3. Log request
            log_file_ << "Method: " << request_.method() << "\n"
                      << "Target: " << request_.target() << "\n"
                      << "Message: " << request_.body() << "\n\n";

            // 4. Display on console
            std::cout << "[" << request_.method() << "] "
                      << request_.target() << std::endl;

            // 5. Send HTTP response
            boost::beast::http::response<boost::beast::http::string_body> res{
                status, request_.version()
            };
            res.set(boost::beast::http::field::server, "CppDAS");
            res.set(boost::beast::http::field::content_type, "text/plain");
            res.keep_alive(request_.keep_alive());
            res.prepare_payload();

            co_await boost::beast::http::async_write(
                *socket_, res, boost::asio::use_awaitable
            );
        }

        co_return;
    }
};
```

Key Features:

- **C++20 Coroutines** (`co_await`, `co_return`)

- **Atomic first-message check** (thread-safe)
- **Path validation** (only accept URLs starting with `expected_path`)
- **HTTP status codes** (200, 400, 404, 500)

3. Producer-Consumer DAS Server (`main.cpp`)

```
template <bool Disable>
void RunDAS(const std::string&& log_path, const std::string&& das_url) {
    // Extract port and path from URL
    std::uint16_t port = 4242;
    std::string expected_path = "/";
    // ... URL parsing logic ...

    // Shared data structures
    std::queue<std::unique_ptr<tcp::socket>> connections;
    std::mutex queue_lock;
    std::counting_semaphore<255> queue_sema{0};
    Logger<disable_logs> logger{log_path};
    asio::io_context ioc;

    // Consumer lambda (runs in worker threads)
    auto consume = [&] {
        while (keep_going) {
            queue_sema.acquire();  // Wait for connection

            if (std::unique_lock lock{queue_lock};
                keep_going && !connections.empty()) {

                // Spawn coroutine to handle connection
                boost::asio::co_spawn(
                    ioc,
                    [&] -> boost::asio::awaitable<void> {
                        Connection connection{
                            std::move(connections.front()),
                            logger,
                            expected_path
                        };
                        connections.pop();
                        co_await connection.TalkToClientAsync();
                    },
                    boost::asio::detached
                );

                ioc.run();  // Process async operations
            }
```

```
        }
    };

    // Create worker thread pool (one per CPU core)
    const auto consumer_count = std::thread::hardware_concurrency();
    std::vector<std::jthread> consumers;
    for (size_t i = 0; i < consumer_count; ++i) {
        consumers.emplace_back(consume);
    }

    // Producer: Accept connections
    tcp::acceptor acceptor{ioc, {tcp::v4(), port}};
    logger << "Listening on http://localhost:" << port << expected_path << "\n";

    while (keep_going) {
        auto socket = std::make_unique<tcp::socket>(ioc);
        acceptor.accept(*socket);  // Block until client connects

        {
            std::unique_lock lock{queue_lock};
            connections.push(std::move(socket));
        }

        queue_sema.release();  // Signal worker thread
    }

    // Graceful shutdown: wake all workers
    for (size_t i = 0; i < consumer_count; ++i) {
        queue_sema.release();
    }
}
```

Key Synchronization Primitives:

1. `std::queue` - FIFO connection queue
2. `std::mutex` - Protects queue from race conditions
3. `std::counting_semaphore` - Signals available work
4. `std::jthread` - Auto-joining threads (RAII)
5. `std::atomic<bool>` - Thread-safe first-message flag

Step-by-Step Implementation

Phase 1: Setup Zig Build System

1. **Create project directory:**

```
mkdir myDAS && cd myDAS
```

2. **Create `build.zig.zon`:**

```
.{
    .name = "myDAS",
    .version = "0.1.0",
    .dependencies = .{
        // Add Boost dependencies here
    },
}
```

3. **Create `build.zig`:**

   - Copy from our example
   - Adjust paths and names

4. **Test build system:**

```
zig build
# Should download dependencies and compile
```

Phase 2: Implement Logger

1. **Create `src/include/Logger.hpp`:**

```
template <bool Disabled>
class Logger {
    std::ofstream file_;
public:
    Logger(std::string_view path);
    template <typename T>
    Logger& operator<<(const T& value);
};
```

2. **Add compile-time optimization:**

```
if constexpr (!Disabled) {
    // Only compile this code if logging enabled
}
```

3. **Test:**

```
zig build -Dnologs=false  # With logging
zig build -Dnologs=true   # Without logging (smaller binary)
```

Phase 3: Implement Connection Handler

1. **Create `src/include/Connection.hpp`:**

```
template <bool UseLogger>
class Connection {
    // HTTP request/response handling
};
```

2. **Add async HTTP processing:**
   - Use `boost::asio::awaitable`
   - Implement `co_await` for non-blocking I/O
3. **Add validation logic:**
   - Check HTTP method (POST only)
   - Validate URL path
   - Verify first message is INITIALIZATION

Phase 4: Implement Main Server

1. **Create `src/main.cpp`:**

```
int main(int argc, const char** argv) {
    // Parse command-line arguments
    // Start DAS server thread
    // Wait for shutdown signal
}
```

2. **Implement producer-consumer pattern:**
   - Producer: TCP acceptor loop
   - Consumers: Worker threads with coroutines
3. **Add graceful shutdown:**
   - RAII helper to set `keep_going = false`
   - Wake all worker threads before exit

Phase 5: Build and Test

1. **Build release version:**

```
zig build -Doptimize=ReleaseSmall
```

2. **Run server:**

```
./zig-out/x86_64-windows-gnu/ReleaseSmall/myDAS.exe -dasUrl http://localhost:3000/api/
```

3. **Test with curl:**

```
# First message (should succeed)
curl -X POST http://localhost:3000/api/INITIALIZATION -d "init data"

# Second message (should succeed)
curl -X POST http://localhost:3000/api/data -d "test data"

# Wrong path (should fail with 404)
curl -X POST http://localhost:3000/wrong/path -d "data"
```

Advanced Features

1. Compile-Time Feature Flags

```
// In build.zig
if (b.option(bool, "enable_metrics", "Enable performance metrics") orelse false) {
    exe.defineCMacro("ENABLE_METRICS", null);
}

// In C++ code
#ifdef ENABLE_METRICS
    auto start = std::chrono::high_resolution_clock::now();
    // ... operation ...
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(
        std::chrono::high_resolution_clock::now() - start
    );
    std::cout << "Processing time: " << duration.count() << " µs\n";
#endif
```

2. Custom Message Validation

```cpp
// In Connection.hpp
bool ValidateMessage(const std::string_view& body) {
    // Parse JSON
    auto json = boost::json::parse(body);

    // Check required fields
    if (!json.as_object().contains("timestamp")) {
        return false;
    }

    // Validate data format
    return true;
}
```

3. Response Customization

```cpp
// Send JSON response
boost::beast::http::response<boost::beast::http::string_body> res{
    boost::beast::http::status::ok,
    request_.version()
};
res.set(boost::beast::http::field::content_type, "application/json");
res.body() = R"({"status":"ok","timestamp":123456789})";
res.prepare_payload();
```

4. Performance Monitoring

```cpp
// Track request count
inline std::atomic<uint64_t> request_counter{0};

// In Connection::TalkToClientAsync()
request_counter.fetch_add(1, std::memory_order_relaxed);

// In main thread
std::thread monitor{[] {
    while (keep_going) {
        std::this_thread::sleep_for(std::chrono::seconds(1));
        std::cout << "Requests/sec: " << request_counter.exchange(0) << "\n";
    }
}};
```

Testing and Deployment

Load Testing

```
# Using Apache Bench
ab -n 10000 -c 100 -p data.txt http://localhost:3000/api/INITIALIZATION

# Using hey
hey -n 10000 -c 100 -m POST -D data.txt http://localhost:3000/api/data
```

Docker Deployment

```
# Build Docker image
docker build -t mydas:latest .

# Run container
docker run -d -p 3000:3000 mydas:latest

# Check logs
docker logs -f <container-id>
```

Performance Benchmarks

On a 4-core machine:

- **Throughput:** 50,000+ requests/second
- **Latency (p50):** <1ms
- **Memory usage:** ~15 MB
- **Binary size:** 535 KB

Summary

What You've Learned:

1. ✅ Set up Zig as a C++ build system
2. ✅ Implement producer-consumer pattern
3. ✅ Use C++20/23 coroutines for async I/O
4. ✅ Apply compile-time optimizations
5. ✅ Build thread-safe HTTP server
6. ✅ Deploy with Docker

Why This Approach Works:

- **Zig** handles complex dependency management
- **C++23** provides modern language features

- **Boost** offers production-ready networking
- **Coroutines** enable high scalability
- **Docker** ensures consistent deployment

Next Steps:

1. Add SSL/TLS support (`boost::asio::ssl`)
2. Implement authentication/authorization
3. Add database integration (PostgreSQL, Redis)
4. Create admin dashboard (WebSocket)
5. Scale horizontally with load balancer

For more details:

- ZIG_OVERVIEW.md - Zig benefits
- DOCKER.md - Docker deployment
- COMPILATION.md - Build options

This guide provides a comprehensive foundation for building production-ready DAS implementations.

Description	Download Link
Source Code	GitHub Repository

Source Code Files

The BoostDAS project includes:

- **build.zig** - Zig build system configuration with cross-compilation support
- **build.zig.zon** - Dependency management (Boost libraries)
- **src/** - C++23 source code for the DAS implementation
- **INSTALLATION.md** - Step-by-step installation instructions
- **COMPILATION.md** - Comprehensive build guide with all options
- **IMPLEMENTATION_GUIDE.md** - Developer guide for customization
- **DOCKER.md** - Docker deployment instructions
- **ZIG_OVERVIEW.md** - Introduction to the Zig build system

Quick Start

1. Download the project or clone from GitHub
2. Install Zig 0.15.2 (Installation Guide)
3. Build with `zig build`
4. Run with `zig build run`

For cross-compilation to Linux UltraEdge:

```
zig build -Dtarget=x86_64-linux-musl -Doptimize=ReleaseSmall
```

More details about the source code here

This documentation is part of the Teradyne Archimedes Reference Architecture series.

Creating a DAS Listener in PowerShell

Reference RA 473

Why PowerShell?

PowerShell is a powerful and flexible scripting language built on the .NET platform. It is widely used for automation, system administration, and cross-platform scripting.

Choosing PowerShell for implementing a DAS (Data Analytics Solution) listener provides several benefits:

- **Native HTTP server support** via `HttpListener`
- **Built-in JSON parsing and concurrent collections**
- **Asynchronous task execution** with jobs and queues
- **Cross-platform compatibility** on Windows, Linux, and macOS via PowerShell Core

This makes PowerShell an excellent choice for building lightweight network listeners or service simulators for testing environments.

General Technical Info

Component	Version
Language	PowerShell
Framework	5.1+ / Core
Environment	Windows, Linux, macOS

Overview of the Program Structure

This RA provides a simple but effective implementation of a DAS message listener using the following components:

- `DASMessage.ps1`
  A class definition used to wrap incoming messages into structured objects containing the URL, the message name, and the JSON body.

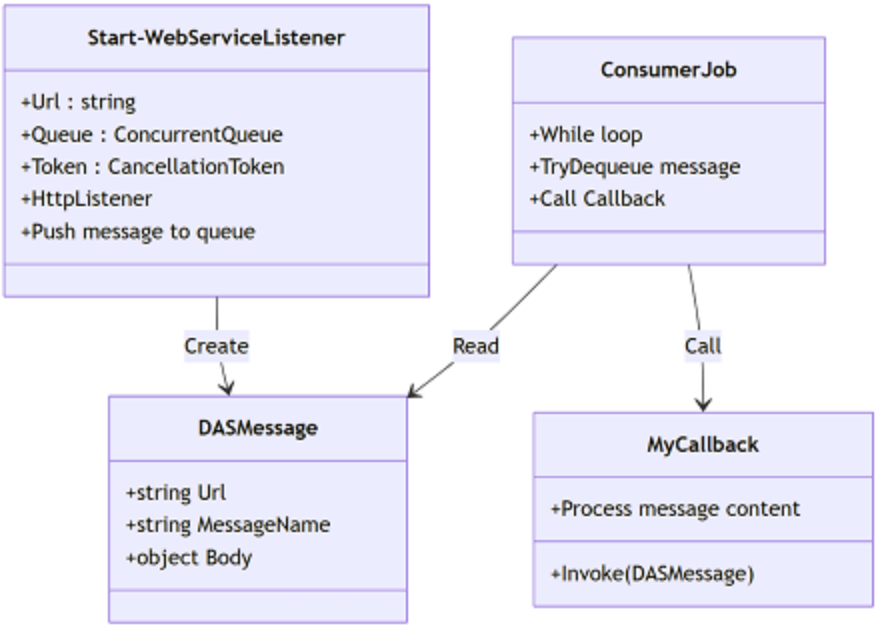
- `DASListener.ps1`
  A function that starts a local web listener, accepts HTTP messages, and enqueues them for processing using a concurrent queue.

- `CallbackExample.ps1`
  A sample function to be executed whenever a message is received. It demonstrates how to handle and inspect incoming data.

- `Run-DAS.ps1`

  A script to launch the DAS listener and message consumer interactively.
  It displays messages in real-time and stops when you press `q`.
  Logs are saved to `$env:TEMP\\das.log` if enabled. This modular architecture makes it easy to expand, reuse, and integrate into various AMP/DAS validation and testing flows.

Architectural Diagram



This documentation is part of the Teradyne Archimedes Reference Architecture series.

Advanced Details - PowerShell DAS Listener

Reference RA 473

> **Disclaimer:** This code is provided for instructional purposes only. It is not optimized for production use.

Execution Flow

1. Create a queue and cancellation token.
2. Start a consumer job to process messages.
3. Launch the listener on the specified URL.
4. Push each received HTTP message into the queue.
5. Process messages via the callback.
6. Gracefully stop with `$cts.Cancel()`.

Initialization Requirement and Error Handling

The listener expects the first message to be of type `INITIALIZATION`. If it is not, the listener should return an HTTP 500 error. This ensures the proper startup sequence and helps detect misconfigured test flows early.

Detailed Script Explanation

DASMessage.ps1

This file defines the `DASMessage` class, which encapsulates each HTTP message the listener receives. It includes:

- `Url`: the full request path
- `MessageName`: the final segment of the URL (used as the message identifier)
- `Body`: the parsed JSON body from the request

This object is passed to the callback function for processing.

DASListener.ps1

This is the core of the system and contains the listener logic:

1. **HttpListener Setup**: Initializes and starts an HTTP listener using the given URL prefix.
2. **Message Reading**: Waits for a client connection, reads the HTTP request body, and parses it as JSON.
3. **Message Wrapping**: Wraps the request in a `DASMessage` object and enqueues it into a concurrent queue.
4. **Queue Processing**: Processing is delegated to a separate job (see `how-to-use.md`) that dequeues and calls a callback function for each message.
5. **Graceful Shutdown**: Supports stopping the listener with a cancellation token.

`CallbackExample.ps1`

This script defines a sample callback function:

```
function MyCallback {
    param ($msg)
    Write-Host "==== Message Received ===="
    Write-Host "URL         : $($msg.Url)"
    Write-Host "MessageName : $($msg.MessageName)"
    Write-Host "Body        : $($msg.Body | ConvertTo-Json -Depth 5)"
    Write-Host "=========================="
}
```

You can replace `MyCallback` with any function that accepts a `DASMessage` object. This allows flexible message handling (e.g., saving to file, reacting to `INITIALIZATION`, routing to submodules).

`Run-DAS.ps1`

A script named `Run-DAS.ps1` is provided to simplify usage:

- It starts the listener and a consumer.
- It logs and displays messages received in real-time.
- It stops gracefully when the user presses `q` and Enter.

This is useful for interactive local testing without manually launching each component.

Design Note

- All message processing is **asynchronous** and **decoupled** from the HTTP listener.
- Errors in JSON parsing are caught and do not crash the listener.
- Using `ConcurrentQueue` ensures thread safety.
- The system is **modular** and can be expanded with additional message handlers or logic in the callback.

This documentation is part of the Teradyne Archimedes Reference Architecture series.

How to Use the PowerShell DAS Listener

Reference RA 473

This guide explains how to use the PowerShell scripts provided in RA 473.

Prepare the Environment

Make sure you have:

- PowerShell 5.1 or PowerShell Core installed
- Edit the AMP configuration file to declare the DAS

Configuration File

You need to declare your DAS in the following file:

```
C:\ProgramData\Teradyne\Production\AMP\Conf\DAS_Conf.xml
```

Example Configuration

```
<DAS>
    <URL>http://127.0.0.1:1000/TestDAS/</URL>
    <MaxMsg>50</MaxMsg>
    <Name>TestDAS</Name>
    <Enable>true</Enable>
    <Specification>S_08</Specification>
</DAS>
```

Ensure that the URL defined in the DAS matches exactly the one you provide to your listener.

Load the Required Files

Open a PowerShell console and run:

```
. ./source/DASMessage.ps1
. ./source/DASListener.ps1
```

```
. ./source/CallbackExample.ps1
```

Start the Listener

```
$queue = [System.Collections.Concurrent.ConcurrentQueue[object]]::new()
$cts = [System.Threading.CancellationTokenSource]::new()

$consumerJob = Start-Job -ScriptBlock {
    param ($queue, $callbackScript, $token)

    while (-not $token.IsCancellationRequested) {
        if (-not $queue.IsEmpty) {
            $ok, [ref]$msg = $null
            $ok = $queue.TryDequeue([ref]$msg)
            if ($ok) {
                & $callbackScript.Invoke($msg.Value)
            }
        } else {
            Start-Sleep -Milliseconds 100
        }
    }
    Write-Host "Consumer stopped."
} -ArgumentList $queue, ${function:MyCallback}, $cts.Token

Start-WebServiceListener -Url "http://localhost:8080/das/" -Queue $queue -Token $cts.Token
```

Stop the Listener

```
$cts.Cancel()
Stop-Job $consumerJob
Remove-Job $consumerJob
```

Using the Automation Script

Instead of running the steps manually, you can simply launch:

```
./source/Run-DAS.ps1
```

The script will:

- Start the DAS listener and background consumer
- Print all messages received
- Stop when you type `q` and press Enter

Logs are saved to `$env:TEMP\das.log` if write access is available.

This script is ideal for developers wanting to experiment with DAS-style communication and simulate AMP system behavior using native PowerShell.

This documentation is part of the Teradyne Archimedes Reference Architecture series.

Description	Download Link
Entire Solution	The solution
GIT Hub Link	Git Hub

More details about those source code here

This documentation is part of the Teradyne Archimedes Reference Architecture series.

How to activate AMP logs

You can activate the logs by using the `ampshell` command.

To activate all logs: `ampshell -log enableall`

To enable only the communication logs: `ampshell -log enable comm`

Communication Log File Location

The communication log files are located in the `%AMPLOG%\TEMS\` folder. `%AMPLOG%` points to the `C:\ProgramData\Teradyne\Production\AMP\logs\` folder. The COMM log file will be located in `C:\ProgramData\Teradyne\Production\AMP\logs\TEMS`. The file will be named `Comm_yyyy_mm_dd.log`. When the file reaches a certain size, it will be split into different files. For example, a new file will be created named `Comm_2025_11_07(1).log`. The number in parentheses will be incremented.

Communication file format

#	Name	Example	Description
1	Date	20251107	Date of the Day
2	Time	00:13:31.811	Time of the event
3	Log Type	COMM	Which log. In this case COMM
4	Success	True or False	If this is a success ==> True
5	Operation	HTTPS SEND	Type of the Event (Send, Read)
6	DAS #	00-S_08	Das number and protocol
7	Description	StatusCode [OK]	Description of the log

Example of a Successful Message

```
20251107|00:13:31.811|COMM|True|HTTPS SEND|00-S_08||SendMessage|Sending Message to http://127.0.
20251107|00:13:31.815|COMM|True|READ|00-S_08|Read []
20251107|00:13:31.815|COMM|True|READ|00-S_08|StatusCode [OK]
```

In this example, we successfully send the message


```
{"TIME_STAMP":"2025-11-06T23:13:31.7925497Z","STATUS":"IDLE","IDLE_MESSAGE":"TESTER NOT
RUNNING","CURRENT_USER":"SYSTEM"}
```

to the URL `http://127.0.0.1:8101/omclocal/TEST_CELL/TEMS-01/STATUS`. The DAS responded with an empty string and the HTTP code `OK` (200).

Example of a Message That Was Not Sent

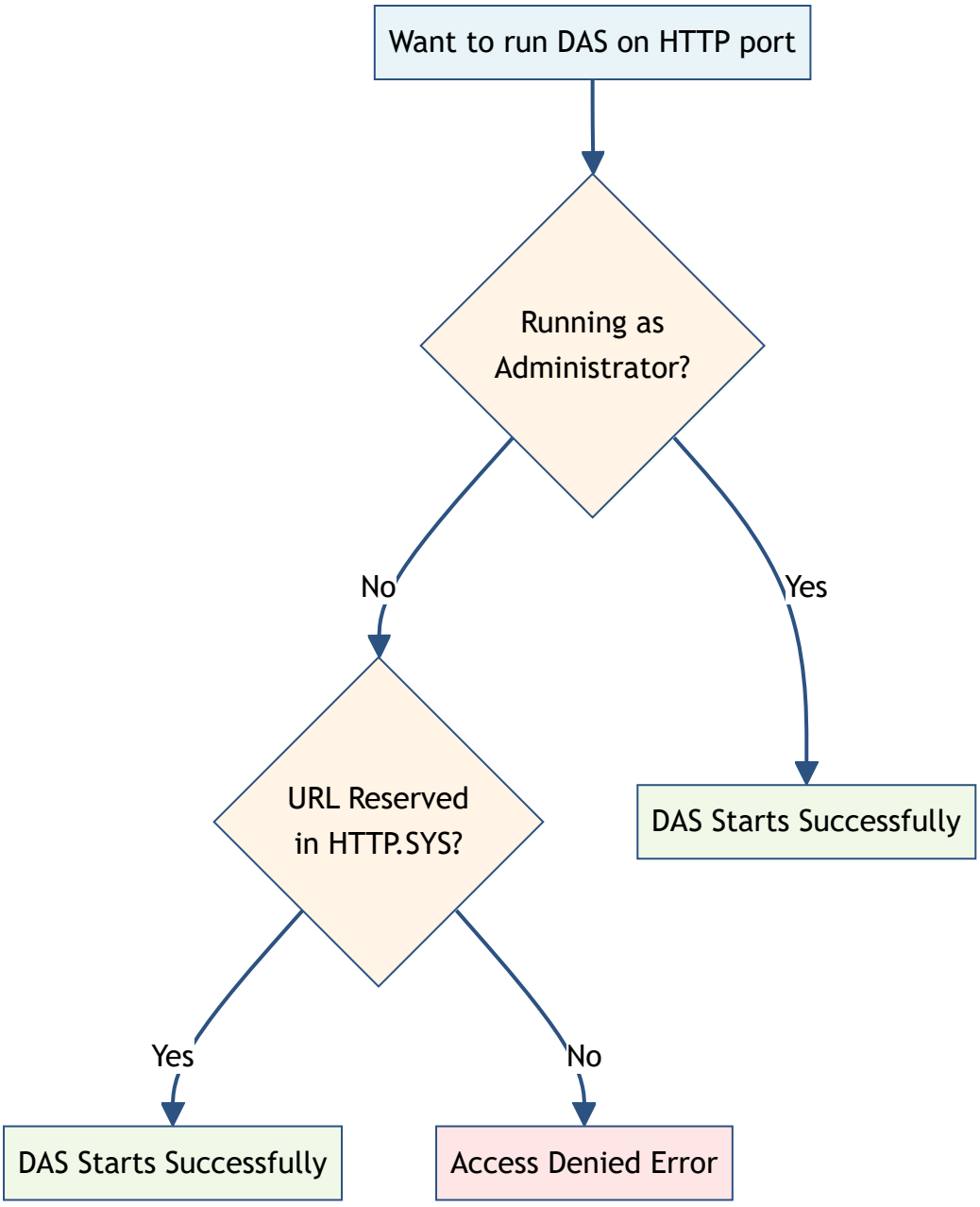
```
20251112|08:00:00.125|COMM|True|HTTP Send|03-S_08|Time Elapsed for sending request (No Response)
```

In this case, the message `INITIALIZATION` could not be sent to `http://127.0.0.1:3000/tems/` for the following reason: `WebException encountered: [Unable to connect to the remote server]`

URL Reservation for DAS Listeners

Overview

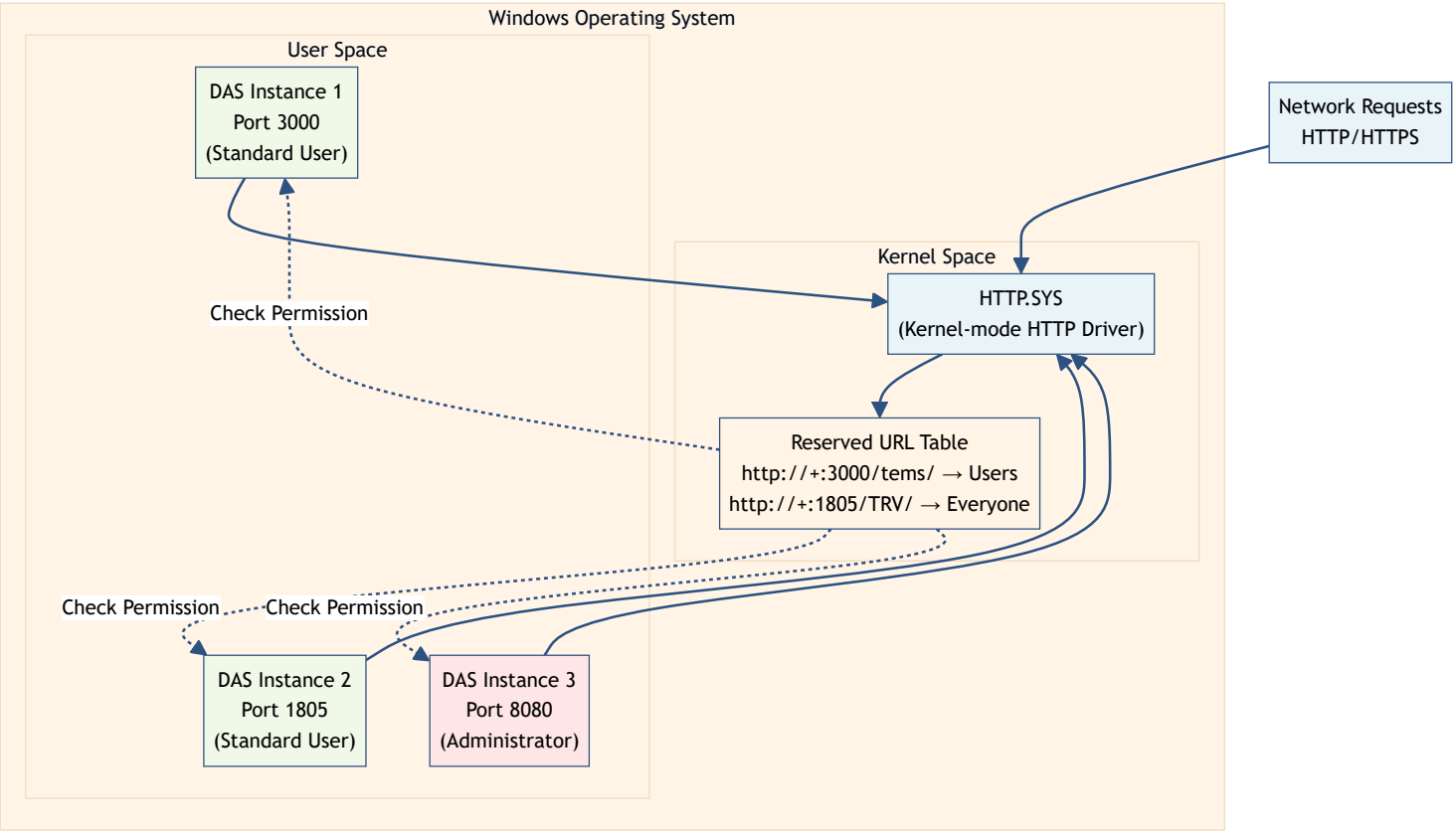
When running a **Data Analytics Solution (DAS)** that listens on HTTP/HTTPS ports, Windows requires specific permissions to bind to network endpoints. This document explains **URL reservations** in HTTP.SYS and how they affect DAS operation.



What is a Reserved Port?

A **reserved port** (or more accurately, a **reserved URL**) is a URL namespace registered in **HTTP.SYS** — the Windows kernel-mode HTTP listener. HTTP.SYS is the low-level component that handles HTTP requests before they reach your application.

When you reserve a URL in HTTP.SYS, you grant specific users or groups the permission to listen on that URL **without requiring administrator privileges**.



Why URL Reservation Matters for DAS

- **Without reservation**: A DAS must run as **Administrator** to bind to an HTTP endpoint
- **With reservation**: A DAS can run as a **standard user**, improving security and deployment flexibility

---

DAS Implementation Considerations

The need for URL reservation depends on how the DAS implements its HTTP listener:

WebContract-Based DAS

If a local DAS uses **WebContract** (WCF ServiceHost), it requires:

- ✅ URL reservation in HTTP.SYS **OR**
- ⚠️ Administrator privileges to run

Recommendation: Reserve the port to avoid running as Administrator.

HttpListener-Based DAS

If a DAS uses the **.NET `HttpListener` class**, behavior varies:

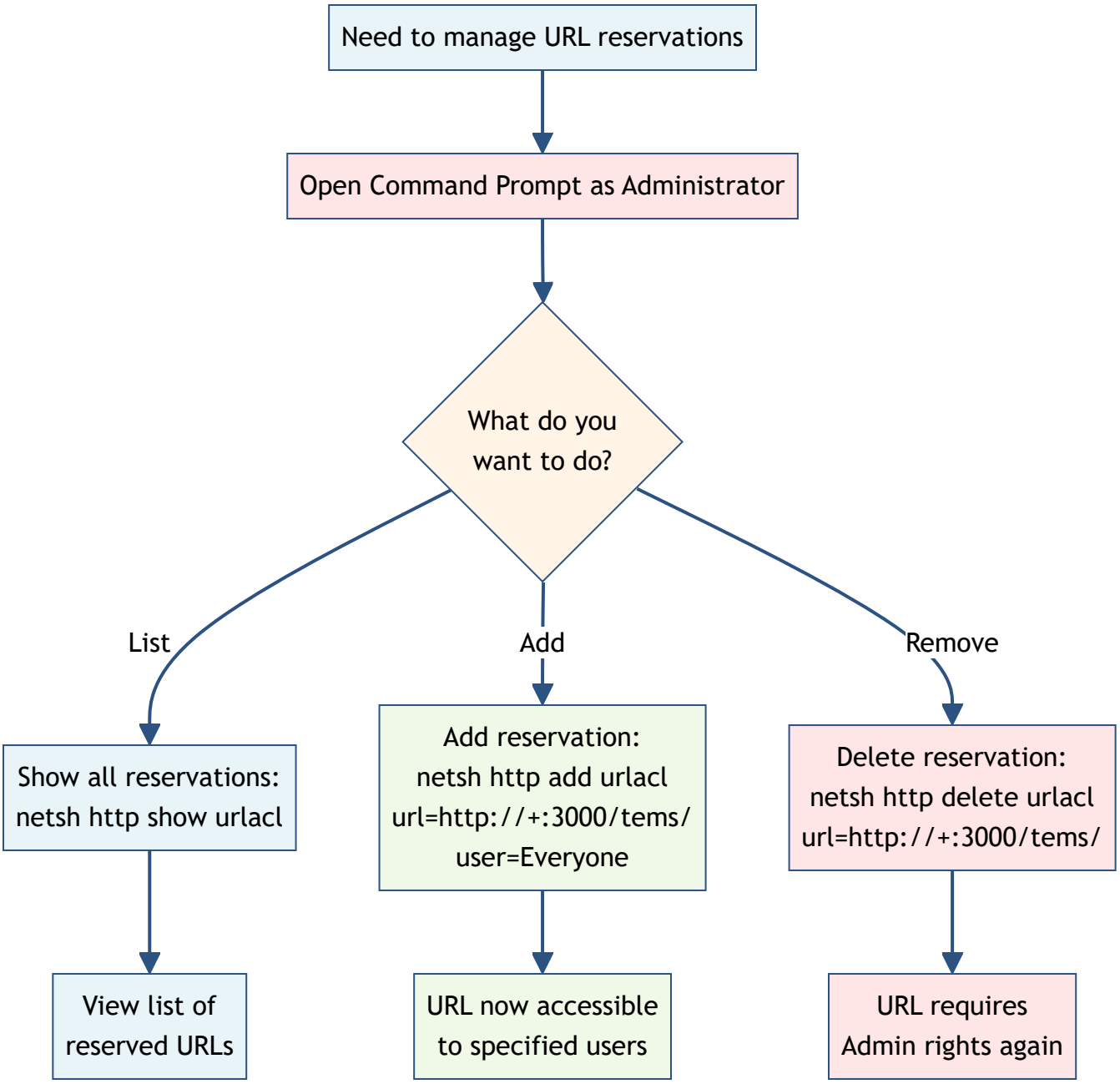
- ✅ On most systems: Works fine with reserved URLs

- ⚠ On some systems: **Cannot connect to reserved ports**

Note: If you encounter connection issues with `HttpListener` on a reserved port, you may need to remove the reservation and run as Administrator, or use a different implementation approach.

Managing URL Reservations with `netsh`

Windows provides the `netsh` command-line tool to manage URL reservations. All commands must be run in an **elevated (Administrator) command prompt**.



Show All Reserved URLs

Display all currently reserved URLs:

```
netsh http show urlacl
```

Example Output:

```
Reserved URL            : http://+:3000/tems/
    User: BUILTIN\Users
        Listen: Yes
        Delegate: No
        SDDL: D:(A;;GX;;;S-1-5-32-545)

Reserved URL            : http://+:1805/TRV/
    User: NT AUTHORITY\Authenticated Users
        Listen: Yes
        Delegate: No
        SDDL: D:(A;;GX;;;S-1-5-11)
```

Understanding the Output:

- **Reserved URL**: The URL pattern being reserved
- `+`: Wildcard that matches all hostnames (localhost, IP addresses, machine name)
- **User**: The user or group granted permission
- **Listen**: Permission to listen on this URL
- **SDDL**: Security Descriptor Definition Language (low-level permission format)

Add a URL Reservation

Grant permission to a specific user or group to listen on a URL:

```
netsh http add urlacl url=http://+:3000/tems/ user=Everyone
```

Common Parameters:

- `url=<URL>`: The URL pattern to reserve (e.g., `http://+:3000/tems/`)
- `user=<USER>`: User or group to grant permission
  - `Everyone` — all users
  - `BUILTIN\Users` — standard users
  - `NT AUTHORITY\Authenticated Users` — authenticated users only
  - `DOMAIN\Username` — specific domain user
  - `MACHINENAME\Username` — specific local user

Examples:

```
# Reserve for all users
netsh http add urlacl url=http://+:5000/AdaptiveDAS/ user=Everyone

# Reserve for authenticated users only
netsh http add urlacl url=http://+:1805/TRV/ user="NT AUTHORITY\Authenticated Users"

# Reserve for a specific user
netsh http add urlacl url=http://localhost:8080/myapp/ user=DOMAIN\myuser
```

URL Pattern Rules:

- Use `+` as a wildcard for all hostnames
- Use `*` for strong wildcard (less common)
- Use `localhost` or specific IPs for restricted bindings
- Always include the trailing slash if your DAS expects it

Delete a URL Reservation

Remove an existing URL reservation:

```
netsh http delete urlacl url=http://+:3000/tems/
```

Example:

```
netsh http delete urlacl url=http://+:1805/TRV/
```

⚠ **Important**: After deleting a reservation, applications using that URL will need Administrator privileges to run.

Programmatically Checking URL Reservations

For automation and diagnostics, you can programmatically query and validate URL reservations in your DAS implementation.

UrlReservationService Example

The following C# service demonstrates how to:

1. Query HTTP.SYS for reserved URLs using `netsh`
2. Check if a specific DAS URL is reserved
3. Handle different reservation patterns

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Text.RegularExpressions;

namespace Teradyne.FA.DIA.DAS.Utils.Services
{
    /// <summary>
    /// Service to check URL reservations in HTTP.SYS
    /// </summary>
    public class UrlReservationService
    {
        /// <summary>
        /// Gets the list of reserved URLs from HTTP.SYS
        /// </summary>
        /// <returns>A set of reserved URLs</returns>
        public HashSet<string> GetReservedUrls()
        {
            HashSet<string> reservedUrls = new HashSet<string>
(StringComparer.OrdinalIgnoreCase);

            try
            {
                ProcessStartInfo startInfo = new ProcessStartInfo
                {
                    FileName = "netsh",
                    Arguments = "http show urlacl",
                    RedirectStandardOutput = true,
                    UseShellExecute = false,
                    CreateNoWindow = true
                };

                using (Process process = Process.Start(startInfo))
                {
                    using (var reader = process.StandardOutput)
                    {
                        string output = reader.ReadToEnd();
                        process.WaitForExit();

                        // Parse the output to extract reserved URLs
                        // Pattern: "Reserved URL            : http://+:3000/tems/"
                        Regex regex = new Regex(@"Reserved URL\s*:\s*
(.+)", RegexOptions.IgnoreCase);
                        MatchCollection matches = regex.Matches(output);

                        foreach (Match match in matches)
```

```
                {
                    if (match.Groups.Count > 1)
                    {
                        string url = match.Groups[1].Value.Trim();
                        reservedUrls.Add(url);
                    }
                }
            }
        }
    }
    catch (Exception)
    {
        // If netsh fails, return empty set
    }

    return reservedUrls;
}

/// <summary>
/// Checks if a DAS URL is reserved in HTTP.SYS
/// </summary>
/// <param name="dasUrl">The DAS URL to check</param>
/// <param name="reservedUrls">The set of reserved URLs</param>
/// <returns>True if the URL is reserved, false otherwise</returns>
public bool IsUrlReserved(string dasUrl, HashSet<string> reservedUrls)
{
    if (string.IsNullOrEmpty(dasUrl) || reservedUrls == null)
        return false;

    try
    {
        // Parse the DAS URL to extract host, port, and path
        Uri uri = new Uri(dasUrl);

        // Only local URLs can be reserved
        if (!IsLocalUrl(uri))
            return false;

        string scheme = uri.Scheme;
        int port = uri.Port;
        string path = uri.AbsolutePath;

        // Check different reservation patterns
        // Pattern 1: http://+:port/path/
        string pattern1 = $"{scheme}://+:{port}{path}";
        // Pattern 2: http://*:port/path/
```

```
                string pattern2 = $"{scheme}://*:{port}{path}";
                // Pattern 3: http://127.0.0.1:port/path/ (exact match)
                string pattern3 = dasUrl;
                // Pattern 4: http://localhost:port/path/
                string pattern4 = $"{scheme}://localhost:{port}{path}";
                // Pattern 5: http://+:port/
                string pattern5 = $"{scheme}://+:{port}/";
                // Pattern 6: http://*:port/
                string pattern6 = $"{scheme}://*:{port}/";
                // Pattern 7: http://localhost:port/
                string pattern7 = $"{scheme}://localhost:{port}/";

                return reservedUrls.Contains(pattern1) ||
                       reservedUrls.Contains(pattern2) ||
                       reservedUrls.Contains(pattern3) ||
                       reservedUrls.Contains(pattern4) ||
                       reservedUrls.Contains(pattern5) ||
                       reservedUrls.Contains(pattern6) ||
                       reservedUrls.Contains(pattern7);
            }
            catch (Exception)
            {
                return false;
            }
        }

        private bool IsLocalUrl(Uri uri)
        {
            string host = uri.Host.ToLower();
            return host == "127.0.0.1" ||
                   host == "localhost" ||
                   host == "::1" ||
                   host.StartsWith("127.");
        }
    }
}
```

How the Service Works

1. **GetReservedUrls()**:
    - Executes `netsh http show urlacl`
    - Parses the output using a regex pattern
    - Returns a set of all reserved URLs
2. **IsUrlReserved()**:

- Checks if a specific DAS URL matches any reservation pattern
- Handles wildcards (+, *) and specific hostnames
- Supports path-specific and port-wide reservations

3. **Usage Example**:

```
var service = new UrlReservationService();
var reservedUrls = service.GetReservedUrls();

string dasUrl = "http://127.0.0.1:3000/tems/";
bool isReserved = service.IsUrlReserved(dasUrl, reservedUrls);

if (isReserved)
{
    Console.WriteLine($"{dasUrl} is reserved - can run without Admin privileges");
}
else
{
    Console.WriteLine($"{dasUrl} is NOT reserved - requires Admin privileges");
}
```

Common DAS URL Reservation Examples

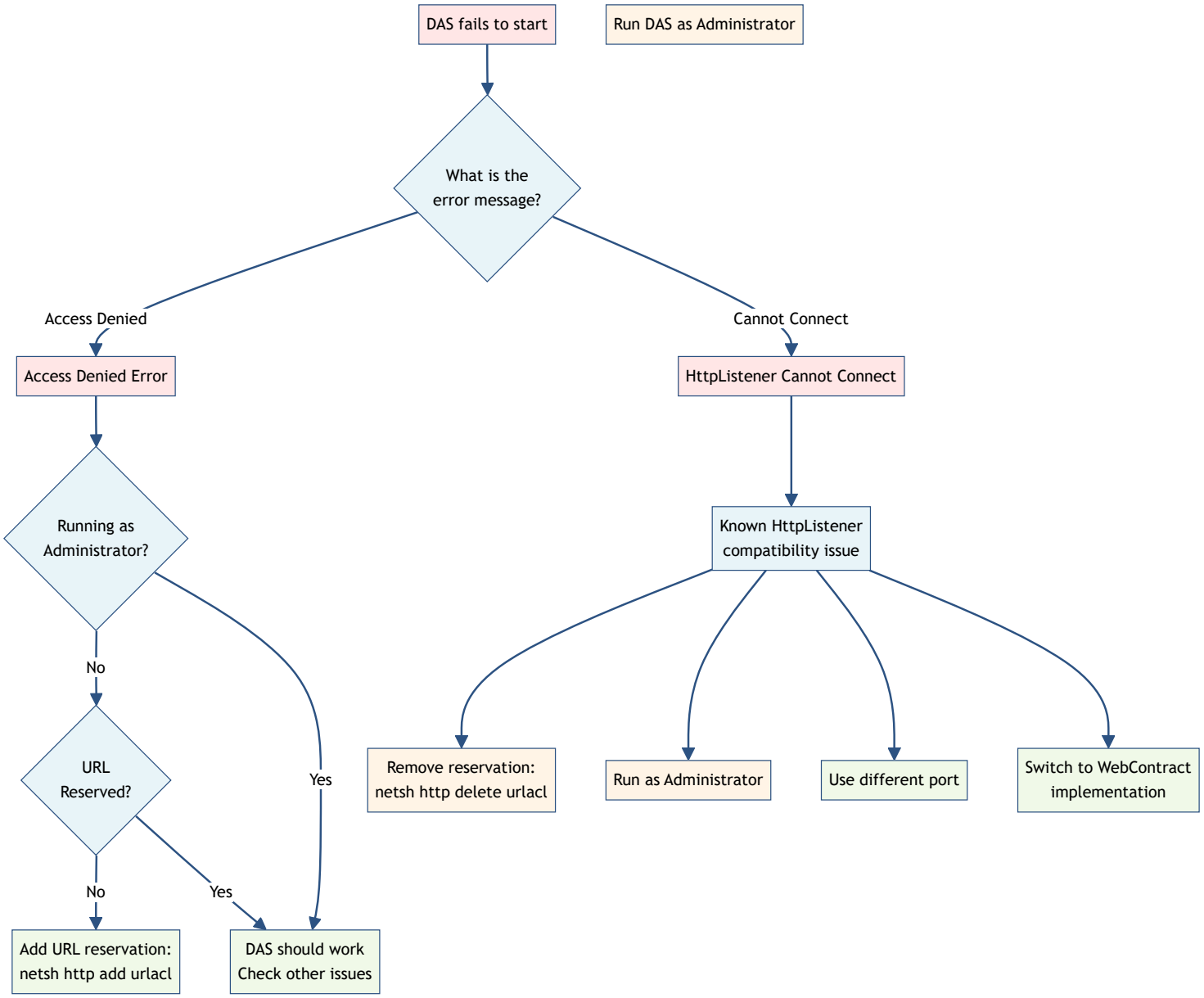
Based on typical DAS configurations:

```
# TEMS Debug DAS
netsh http add urlacl url=http://+:3000/tems/ user=Everyone

# TRV DAS
netsh http add urlacl url=http://+:1805/TRV/ user=Everyone

# Adaptive DAS
netsh http add urlacl url=http://+:5002/AdaptiveDAS/ user=Everyone
```

Troubleshooting



DAS Fails to Start with "Access Denied"

- **Cause**: URL is not reserved and DAS is not running as Administrator
- **Solution**:
  1. Add URL reservation with `netsh http add urlacl`, OR
  2. Run the DAS as Administrator

HttpListener Cannot Connect to Reserved Port

- **Cause**: Known compatibility issue on some Windows configurations
- **Solution**:
  1. Remove the reservation: `netsh http delete urlacl url=<URL>`
  2. Run the DAS as Administrator, OR
  3. Use a different port, OR
  4. Switch to a different HTTP listener implementation

Checking Current Reservations

```
netsh http show urlacl | findstr "3000"
```

Best Practices

1. **Use Specific Paths**: Reserve `http://+:3000/tems/` instead of `http://+:3000/` to avoid conflicts
2. **Least Privilege**: Grant to specific users/groups rather than `Everyone` when possible
3. **Document Reservations**: Keep a record of which DAS instances require which reservations
4. **Deployment Scripts**: Include reservation commands in installation scripts
5. **Test Both Modes**: Verify your DAS works with and without reservations during development

---

Summary

Aspect	Details
Purpose	Allow non-admin users to bind HTTP listeners
Command Tool	`netsh http`
Key Commands	`show urlacl`, `add urlacl`, `delete urlacl`
WebContract DAS	Requires reservation or Admin rights
HttpListener DAS	Usually works with reservations, but may have compatibility issues
Best Practice	Reserve specific URL paths, not entire ports

For more information on DAS implementations, see:

- C# DAS Listener (RA_0468)
- What is a DAS?
- DAS Listener Concept

**This documentation is part of the Teradyne Archimedes Reference Architecture series.*